MULTI-OBJECTIVE OPTIMIZATION FOR ENERGY-AWARE INVERSE KINEMATICS OF ROBOTICS

by

Xibin Zhou

A Thesis

Submitted to the Faculty of Purdue University

In Partial Fulfillment of the Requirements for the degree of

Master of Science in Electrical and Computer Engineering

Department of Electrical and Computer Engineering

Hammond, Indiana

December 2024

THE PURDUE UNIVERSITY GRADUATE SCHOOL
STATEMENT OF COMMITTEE APPROVAL

Dr. Lizhe Tan, Chair

Department of Electrical and Computer Engineering

Dr. Quamar Niyaz

Department of Electrical and Computer Engineering

Dr. Khair A. Al Shamaileh

Department of Electrical and Computer Engineering

Dr. Nasser Houshangi

Department of Electrical and Computer Engineering

Approved by:

Dr. Quamar Niyaz

Dedicated to my parents, Zhili Wang and Zongkui Zhou, who have always supported me.

ACKNOWLEDGMENTS

I would like to first thank my academic advisor and committee chair, Dr. Lizhe Tan, for his patient guidance and constant encouragement throughout my graduate studies. I have learned what it means to be a researcher and how to think like a researcher.

I would also like to thank the committee members, Dr. Quamar Niyaz, Dr. Khair Al Shamaileh, and Dr. Nasser Houshangi, for their dedicated teaching, which helped me lay a solid foundation of technical knowledge.

Also, I would like to thank the Department of Electrical and Computer Engineering for providing me with all the support, including grader, graduate teaching assistant, and research assistant positions, to complete my thesis research.

Additionally, I would like to thank Dr. Chenn Zhou and Kyle Toth for providing a research opportunity at CIVS, where I could engage in several projects and learn from industry professionals.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

BFGS	Broyden–Fletcher–Goldfarb–Shanno
DH	Denavit–Hartenberg
DOF	Degrees of Freedom
FK	Forward Kinematics
FOPID	Fractional-Order Proportional-Integral-Derivative
GA	Genetic Algorithm
IK	Inverse Kinematics
IKP	Inverse Kinematics Problem
LM	Levenberg-Marquardt
MOO	Multi-Objective Optimization
MOOP	Multi-Objective Optimization Problem
PID	Proportional-Integral-Derivative
PSO	Particle Swarm Optimization
RMSE	Root Mean Square Error
UR	Universal Robots
WMOGA	Weighted Multi-Objective Genetic Algorithm
WMOPSO	Weighted Multi-Objective Particle Swarm Optimization

LIST OF SYMBOLS

A	Amplitude
C	cos()
f	Frequency
r	Radius
S	sin()
φ	Phase

ABSTRACT

Inverse kinematics is the fundamental study of robot motion in space. Usually, the control command for a robot is only given by target position and orientation, in which the motions of the joint motors are found from the inverse kinematics. For the specified position or orientation, there are often multiple inverse kinematics solutions. The traditional method obtains all these solutions and then selects one of them according to some rules, such as minimum energy, shortest distance, shortest time, etc. In this thesis research, weighted multi-objective genetic algorithm (WMOGA) and weighted multi-objective particle swarm optimization (WMOPSO) are developed to solve energy-aware inverse kinematics by considering that the energy consumption of the robot is proportional to the total amount of joint rotations to the target pose. Therefore, the objective functions are proposed based on minimizing the target position error, target orientation error, and total amount of joint rotations to obtain the inverse kinematics solution. The algorithms developed in this thesis use a weighted sum of the multiple objective functions to iterate over a certain range of weights to obtain the inverse kinematics solution within the required error range. Furthermore, the proposed algorithm can be applied to the robotic model with any number of degrees of freedom. The trajectories solved by the proposed WMOGA and WMOPSO algorithms are verified by simulations and validated by the physical robotic arm. The achieved inverse kinematic solutions demonstrate a promising performance and have the potential for applications of the inverse kinematics of robotics.

1. INTRODUCTION

1.1 Background and significance

Technologies such as robotics and autonomous driving are gradually entering the daily lives of more and more people. The wish of “a computer on every desk, and in every home” has become everyone having a robot buddy or two [1]. In this process, there are research topics such as large models and embodied intelligence that focus on the interaction between robots and their external environments [2]. Meanwhile, there is also room for improvement in the internal of robots.

With the development of modern industry and automation, numerous robotics companies have emerged, producing a wide variety of robotic arms. These robotic arms can be mainly categorized into two types: industrial robotic arms and collaborative robotic arms. Industrial robotic arms are used in factories to replace human labor, reduce production costs, and improve efficiency. They can also perform dangerous tasks that would otherwise require human intervention. Most of them are designed with a centrosymmetric structure and have a large payload and fast joint rotation speed to meet manufacturing needs. Collaborative robotic arms refer to robots that can interact directly with humans within a collaborative area. In other words, collaborative robotic arms and humans can work in the same area, typically utilizing the UR (Universal Robots) type structure as an example. This thesis focuses on collaborative robotic arms, aiming to study their kinematic models and improve their precision performance with energy awareness.

1.2 Literature review

The concept of modern robotics originated in the 20th century. In 1954, George Devol was granted the patent for Unimate, a robotic arm capable of transporting die-cast parts and welding them in place [3]. In 1961, the first industrial robot in the world, Unimate, was installed on the assembly line at General Motors, marking the birth of industrial robots. Collaborative robots (cobots) were proposed by Northwestern University professors Edward Colgate and Michael Peshkin to assist workers in working more efficiently rather than replacing their jobs, and to reduce the risk of injury [4, 5]. In 2004, KUKA released the first collaborative robot, LBR3 [6]. In 2008, Universal Robots from Denmark delivered their first collaborative robot, UR5, with a payload of

5 kilograms, becoming well-known as UR structure collaborative robots. Today, collaborative robots are rapidly developing, control methods are continuously evolving, and inverse kinematics should be rethought.

The inverse kinematics problem (IKP) is a solvable problem. Its challenge is not what the solution is, but how to find the optimal solution. For a robotic arm, the most common action is to pick and place, and in order to make the end effector reach the target pose more accurately, the inverse kinematics of the robotic arm need to be investigated to solve for the optimal control amount of the robotic arm. Although there are traditional solutions for inverse kinematics, such as analytical solutions and numerical solutions, the analytical solution is usually only applicable to robotic arms with simple geometrical structures or kinematic characteristics, and it becomes very difficult or even impossible to solve the analytical solution for the robotic arms with complex structure or high degrees of freedom [7]. Similarly, the numerical solution requires a lot of computation and time, especially for the high degrees of freedom robotic arms, and the accuracy of the numerical solution depends on the choice of numerical algorithms, such as BFGS (Broyden–Fletcher–Goldfarb–Shanno) and Levenberg-Marquardt (LM) algorithms, which may have numerical errors and convergence problems [8-10]. Additionally, the geometric method is another approach to solving the inverse kinematics. It involves rearranging the vertices of the robotic arm joints in space to find the solution to the IKP [11]. The articles [12-16] transform the IKP into a multi-objective optimization problem (MOOP) using optimization-based or neural network methods such as particle swarm optimization algorithms or reinforcement learning.

However, energy is overlooked in this optimization problem. With the rapid increase in energy consumption due to large models, generative pre-training transformers, and GPU clusters, as well as the rising market share of electric vehicles [17-19], energy issues should be integrated into optimization algorithms as much as possible. The work in [20-24] has applied the concept of energy-aware in operating systems, routing, wireless networks, motion control, and robot control. In inverse kinematics and motor control, energy-aware can still be considered a key factor for solution or optimization, thereby reducing energy consumption and extending the lifetime of motors.

On the other hand, the reduction of robot energy consumption which can be considered as a factor proportional to the total amount of joint rotations to the target pose can also lead to a longer lifespan of the robots in use. This thesis focuses on developing weighted multi-objective genetic

and particle swarm optimization algorithms based on multi-objective optimization (MOO), which incorporates minimizing the total amount of joint rotations to find the inverse kinematics.

1.3 Research methodology

This thesis is focused on solving the inverse kinematics problem. Therefore, the main methodology involves using mathematical tools to evaluate the performance of the algorithms, simulation to verify the results, and finally running the results on a physical robot, before developing the algorithms for solving inverse kinematics, a robotic arm model needs to be created to verify the accuracy of a solution and to enable iterations within the optimization algorithm. In the works of Rokbani et al. [12-14], related algorithms have been attempted for multi-objective optimization to solve the inverse kinematics of robotic arms, achieving high accuracy in obtaining specified positions and orientations for the end effector. This thesis introduces energy-aware into the inverse kinematics problem by considering that the energy consumption of the robot is proportional to the total amount of joint rotations to the target position. Based on this, the total amount of joint rotations is incorporated as a third objective function and iterated alongside the others to investigate the balance between precision and energy consumption in inverse kinematics.

The proposed energy-aware algorithms can first be implemented in simulation environments to evaluate their performance. Then testing on the real robotic arm can also be conducted to validate algorithms under practical conditions.

1.4 Thesis organization

The thesis is structured into five chapters, including this introductory chapter.

Chapter 2 is an overview of the methods used in the study of the inverse kinematics problem and the derivation of the relevant formulas. It includes the derivation of the forward kinematics of the robotic arm used, visualizing the structure model in MATLAB, and establishing a forward kinematics model function to verify the inverse kinematics results.

Chapter 3 introduces two optimization algorithms and their implementation steps for the inverse kinematics multi-objective optimization based on previous work to develop the energy-aware algorithms, that is, weighted multi-objective genetic algorithm (WMOGA) and weighted multi-objective particle swarm optimization (WMOPSO).

Chapter 4 presents two groups of experiments and uses different robotic arm platforms, as well as evaluates the performance of the algorithms.

Finally, Chapter 5 summarizes the work presented in this thesis, including future work.

2. ENERGY-AWARE INVERSE KINEMATICS IN ROBOTICS

2.1 Introduction

This chapter defines and analyzes the problem of energy-aware inverse kinematics. It begins with the derivation of forward kinematics, using the standard Denavit-Hartenberg (DH) method to establish the Cartesian coordinate system for the collaborative robotic arms studied in this thesis and deriving the corresponding DH parameters, thereby establishing the forward kinematics model. In the inverse kinematics section, the existence of inverse kinematics solutions and the multiple solutions problem are discussed, and an energy-aware function is defined for the inverse kinematics problem, which is a key focus of this chapter. Finally, the Monte Carlo method is used to analyze the two robotic arms studied, which provides the basis for setting target points and creating trajectories in the later experimental section in Chapter 4.

2.2 Forward kinematics

2.2.1 Denavit–Hartenberg parameters

Forward kinematics is prior work in inverse kinematics for a robotic arm with a known rigid body structure. In this thesis, the collaborative robotic arm is taken as the research object, and the Cartesian coordinate system is established to conform to the structure of several collaborative robotic arms currently available on the market, and the standard DH (Denavit–Hartenberg) method is used to derive its DH parameters.

The DH parameter method was proposed by Denavit and Hartenberg in 1955. It is a systematic method to describe the links and joints in a serial link chain. This method is known as the standard DH method, which is described in the textbook by Saeed B. Niku [25]. In 1986, Khalil and Kleinfinger proposed a modified DH parameter, in which the joint coordinate system is established at the proximal end of the link, rather than the distal end, this method is known as the modified DH method, which is adopted in the book by John J. Craig [26].

This thesis focuses on the study of inverse kinematics. Due to the simplicity and generality of the structure of the six degrees of freedom robotic arm under study, the standard DH convention is adopted in this thesis for the establishment of the coordinate system and derivation.

Through a comparative study of the desktop-level robotic arm myCobot 280 Pi from Elephant Robotics [27] and the UR3 robotic arm from Universal Robots [28], the Cartesian coordinate system of the collaborative robotic arm structure is established, as shown in Figure 2.1.

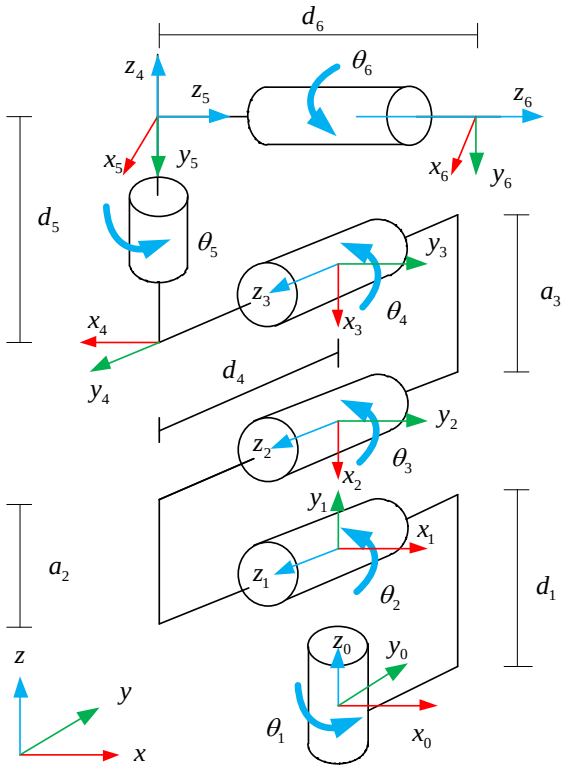


Figure 2.1. Cartesian coordinate system and DH parameters for a 6-DOF robotic arm.

Based on the established coordinate system, the DH parameters obtained using the standard DH method are listed in Table 2.1.

Table 2.1. DH parameters for UR type collaborative robotic arms.

#	θ(rad)	d	a	α
0-1	θ_1	d1	0	π/2
1-2	θ_2-π/2	0	-a2	0
2-3	θ_3	0	-a3	0
3-4	θ_4-π/2	d4	0	π/2
4-5	θ_5+π/2	d5	0	-π/2
5-6	θ_6	d6	0	0

2.2.2 Forward kinematics modeling

The collaborative robotic arms studied in this thesis can share a common set of DH parameters with different joint distances due to the same degrees of freedom and similar structures. The two robotic arms studied are displayed in Figure 2.2.

(a)

(b)

Figure 2.2. Robotic Arms. (a) myCobot 280 Pi [27]; (b) UR3 [29].

In optimization algorithms, one or more fitness functions are needed to minimize the error for each iteration. In this research, the output of the optimization algorithm will be the joint rotational angles of the robotic arm, therefore it is necessary to establish a forward kinematic model to convert the joint rotations into the position so that the position error can be minimized to find the optimal solution.

The control for the precision of the robotic arm in this study lies in the position and orientation of the end effector. The first step is the derivation of the position of the end effector, which is written in the form of Equation (2.1), where n, o, and a are derived from the words of normal, orientation, and approach, respectively. Since the z-axis coincides with the rotation axis of the rotary joints, the robot approaches the object along the z-axis of the gripper, thus it is called

the approach-axis or a-axis. The orientation in which the gripper approaches the object is called the orientation axis or o-axis. The x-axis is normal to both z- and a-axes, so it is called the normal-axis or the n-axis [25].

$$F = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.1}$$

When the DH parameters are known, the transformation T between two frames can be written in Equation (2.2) listed below.

$$A = \begin{bmatrix} \cos\theta & -\sin\theta\cos\alpha & \sin\theta\sin\alpha & a\cos\theta \\ \sin\theta & \cos\theta\cos\alpha & -\cos\theta\sin\alpha & a\sin\theta \\ 0 & \sin\alpha & \cos\alpha & d \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.2}$$

For six degrees of freedom collaborative robotic arm with the DH parameters in Table 2.1 and the rotations of joint θ_1 to θ_6, and by substituting them into Equation (2.2), respectively, Equations (2.3) to (2.8) can be obtained.

$$A_1 = \begin{bmatrix} \cos\theta_1 & 0 & \sin\theta_1 & 0 \\ \sin\theta_1 & 0 & -\cos\theta_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.3}$$

$$A_2 = \begin{bmatrix} \sin\theta_2 & \cos\theta_2 & 0 & -a_2\sin\theta_2 \\ -\cos\theta_2 & \sin\theta_2 & 0 & a_2\cos\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.4}$$

$$A_3 = \begin{bmatrix} \cos\theta_3 & -\sin\theta_3 & 0 & -a_3\cos\theta_3 \\ \sin\theta_3 & \cos\theta_3 & 0 & -a_3\sin\theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.5}$$

$$A_4 = \begin{bmatrix} \sin\theta_4 & 0 & -\cos\theta_4 & 0 \\ -\cos\theta_4 & 0 & -\sin\theta_4 & 0 \\ 0 & 1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.6}$$

$$A_5 = \begin{bmatrix} -\sin\theta_5 & 0 & -\cos\theta_5 & 0 \\ \cos\theta_5 & 0 & -\sin\theta_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.7}$$

$$A_6 = \begin{bmatrix} \cos\theta_6 & -\sin\theta_6 & 0 & 0 \\ \sin\theta_6 & \cos\theta_6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.8}$$

Thus, the transformation ${}^0T_6$ from the base of the robotic arm to the end effector can be calculated by post-multiplying transformations: A_1 to A_6. The resultant transformation is shown in Equation (2.9).

$$
\begin{aligned}
{}^{0}T_6 &= A_1A_2A_3A_4A_5A_6 \\
&= A_{12}A_{34}A_{56} \\
&= \begin{bmatrix} \cos\theta_1 & 0 & \sin\theta_1 & 0 \\ \sin\theta_1 & 0 & -\cos\theta_1 & 0 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \sin\theta_2 & \cos\theta_2 & 0 & -a_2\sin\theta_2 \\ -\cos\theta_2 & \sin\theta_2 & 0 & a_2\cos\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&\times \begin{bmatrix} \cos\theta_3 & -\sin\theta_3 & 0 & -a_3\cos\theta_3 \\ \sin\theta_3 & \cos\theta_3 & 0 & -a_3\sin\theta_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \sin\theta_4 & 0 & -\cos\theta_4 & 0 \\ -\cos\theta_4 & 0 & -\sin\theta_4 & 0 \\ 0 & 1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&\times \begin{bmatrix} -\sin\theta_5 & 0 & -\cos\theta_5 & 0 \\ \cos\theta_5 & 0 & -\sin\theta_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta_6 & -\sin\theta_6 & 0 & 0 \\ \sin\theta_6 & \cos\theta_6 & 0 & 0 \\ 0 & 0 & 1 & d_6 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} C_1S_2 & C_1C_2 & S_1 & -a_2C_1S_2 \\ S_1S_2 & S_1C_2 & -C_1 & -a_2S_1S_2 \\ -C_2 & S_2 & 0 & a_2C_2+d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} S_{34} & 0 & -C_{34} & -a_3C_3 \\ -C_{34} & 0 & -S_{34} & -a_3S_3 \\ 0 & 1 & 0 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} -S_5C_6 & S_5S_6 & -C_5 & -d_6C_5 \\ C_5C_6 & -C_5S_6 & -S_5 & -d_6S_5 \\ -S_6 & -C_6 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} C_1S_2 & C_1C_2 & S_1 & -a_2C_1S_2 \\ S_1S_2 & S_1C_2 & -C_1 & -a_2S_1S_2 \\ -C_2 & S_2 & 0 & a_2C_2+d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -S_{34}S_5C_6+C_{34}S_6 & S_{34}S_5S_6+C_{34}C_6 & -S_{34}C_5 & -d_6S_{34}C_5-d_5C_{34}-a_3C_3 \\ C_{34}S_5C_6+S_{34}S_6 & -C_{34}S_5S_6+S_{34}C_6 & C_{34}C_5 & d_6C_{34}C_5-d_5S_{34}-a_3S_3 \\ C_5C_6 & -C_5S_6 & -S_5 & -d_6S_5+d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} n_x & o_x & a_x & d_6C_1C_{234}C_5-d_5C_1S_{234}-a_3C_1S_{23}-d_6S_1S_5+d_4S_1-a_2C_1S_2 \\ n_y & o_y & a_y & d_6S_1C_{234}C_5-d_5S_1S_{234}-a_3S_1S_{23}+d_6C_1S_5-d_4C_1-a_2S_1S_2 \\ n_z & o_z & a_z & d_6S_{234}C_5+d_5C_{234}+a_3C_{23}+a_2C_2+d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned}
\tag{2.9}
$$

The orientation elements in (2.9) can be expressed in Equations (2.10) to (2.18).

$$n_x = C_1C_{234}S_5C_6 + C_1S_{234}S_6 + S_1C_5C_6 \tag{2.10}$$

$$n_y = S_1C_{234}S_5C_6 + S_1S_{234}S_6 - C_1C_5C_6 \tag{2.11}$$

$$n_z = S_{234}S_5C_6 - C_{234}S_6 \tag{2.12}$$

$$o_x = -C_1C_{234}S_5S_6 + C_1S_{234}C_6 - S_1C_5S_6 \tag{2.13}$$

$$o_y = -S_1C_{234}S_5S_6 + S_1S_{234}C_6 + C_1C_5S_6 \tag{2.14}$$

$$o_z = -S_{234}S_5S_6 - C_{234}C_6 \tag{2.15}$$

$$a_x = C_1C_{234}C_5 - S_1S_5 \tag{2.16}$$

$$a_y = S_1C_{234}C_5 + C_1S_5 \tag{2.17}$$

$$a_z = S_{234}C_5 \tag{2.18}$$

By replacing the components of the position vector in the homogeneous transformation matrix with p_x, p_y, and p_z respectively, we can rewrite Equation (2.1) as follows:

$$A_{123456} = \begin{bmatrix} n_x & o_x & a_x & d_6a_x - d_5C_1S_{234} - a_3C_1S_{23} + d_4S_1 - a_2C_1S_2 \\ n_y & o_y & a_y & d_6a_y - d_5S_1S_{234} - a_3S_1S_{23} - d_4C_1 - a_2S_1S_2 \\ n_z & o_z & a_z & d_6a_z + d_5C_{234} + a_3C_{23} + a_2C_2 + d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2.19}$$

$$d_6a_x - d_5C_1S_{234} - a_3C_1S_{23} + d_4S_1 - a_2C_1S_2 = p_x \tag{2.20}$$

$$d_6a_y - d_5S_1S_{234} - a_3S_1S_{23} - d_4C_1 - a_2S_1S_2 = p_y \tag{2.21}$$

$$d_6a_z + d_5C_{234} + a_3C_{23} + a_2C_2 + d_1 = p_z \tag{2.22}$$

Based on Equations (2.19) to (2.22), a function with six joint angles as the input and a matrix defined in Equation (2.1) as an output is implemented in MATLAB. By inputting the DH parameter values of the specific robotic arm from Table 2.2 into the model, one can verify the inverse kinematics solution obtained from the inverse kinematics algorithm.

Table 2.2. Specific DH parameters of the robotic arms.

Model	d1 (mm)	d4 (mm)	d5 (mm)	d6 (mm)	a2 (mm)	a3 (mm)
myCobot 280 Pi	131.56	66.39	73.18	43.6	110.4	96
UR3	151.9	112.35	85.35	81.9	243.65	213.25

2.3 Inverse kinematics

2.3.1 Existence of inverse kinematics solutions

The inverse kinematics of a robotic arm is to solve the joint rotational angles that enable the end effector to achieve a given target position and orientation. The existence of an inverse kinematics solution depends on the workspace of a robotic arm, that is, the range of motion in space that the end effector can move to with the robot base fixed. For inverse kinematics, it is not only required that the position vector is in the workspace, but the orientation also has to meet the requirement to be considered as an inverse kinematics solution.

For the kinematic model established in the Cartesian coordinate system, positions are commonly represented by spatial coordinates (x, y, z). As for the representation of orientation, there are various methods, besides the orientation matrix defined in Section 2.1.2, which is the rotation matrix of the end effector, there are also rotation vectors, Euler angles, and quaternions, which can be converted to each other. These representations have their own advantages and disadvantages and are suitable for different application scenarios [30].

The orientation matrix is a 3 by 3 matrix used to describe the orientation of an object in three-dimensional space. Although the orientation matrix is convenient for calculation, it has issues with overparameterization and difficulty in differentiation. The rotation vector (or axis-angle representation) describes rotation using a rotation axis and a rotation angle. The rotation vector has a compact form but exhibits periodicity, thus showing singularities in some cases. Euler angles represent rotation by specifying the rotation angles around three mutually perpendicular axes. This method is intuitive and easy to understand, but it requires specifying the rotation order and can encounter gimbal lock in certain situations. Quaternions are a compact representation method that can describe rotation without singularities. Although using the quaternions is not very intuitive to users, they avoid overparameterization and singularities issues, making them suitable for scenarios requiring smooth interpolation.

In this study, since the output of the forward kinematics model directly contains the orientation matrix, it is preferred for its computational convenience and the reduced risk of errors from conversions. Therefore, the orientation matrix is adopted in this thesis.

2.3.2 Multiple solutions problem

Since solving the inverse kinematics only requires position or orientation, it is easy to lead multiple solutions. In the analytic method, we have to achieve all the solutions and then select one of them according to certain criteria. Common selection criteria include minimum energy, shortest distance, and shortest time. For the minimum energy criterion, a set of joint angles is selected with the lowest energy consumption. The shortest distance is to select a set of solutions where the end effector moves the shortest distance from the initial position to the target position. The shortest time refers to the solution that allows the robotic arm to reach the target position and orientation in the shortest time, which considers the speed and acceleration constraints of the robotic arm joints to achieve the fastest motion [31].

2.3.3 Energy-aware inverse kinematics

Energy-aware inverse kinematics refers to inverse kinematics solutions that minimize energy consumption. Traditional analytical inverse kinematics solutions, when multiple solutions exist, need obtaining all these solutions and then selecting the energy-optimal solution.

In this thesis, the energy-aware inverse kinematics is considered as the energy consumption of the robotic arm while the process of solving the inverse kinematics, allows the robotic arm to reach the target position and orientation in a way that minimizes the amount of joint rotations. For a robotic arm with n revolute joints, its amount of joint rotations, $E(\theta_1,...,\theta_n)$, is defined as Equation (2.23).

$$E(\theta_1,...,\theta_n) = \sum_{i=1}^{n} \left|\theta_n\right| \tag{2.23}$$

In multi-objective optimization, the first and second objective functions can be set as the error functions of target position and orientation, respectively, and the third objective function can be set as a function related to the amount of joint rotations to obtain the energy-aware inverse kinematics solution. Energy-aware inverse kinematics is particularly important in applications where energy efficiency is crucial, such as battery-operated robots, long-duration robotic tasks, and sustainable automation systems.

2.4 Workspace analysis

Since the experimental section in the thesis is to manually set single points and trajectories used to verify the developed algorithms, it is necessary to ensure that all points are within the workspace of the robotic arms before creating the trajectories.

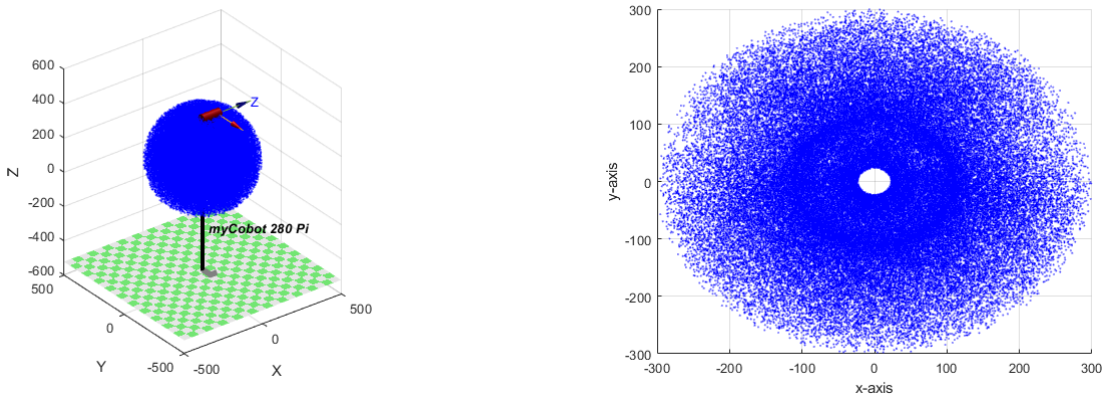
In the process of studying the workspace of the robotic arm, the Robotics Toolbox [32, 33] was used to create the joint and link information of the robotic arm based on the DH parameters in Table 2.1 and the specific model of the robotic arm, as well as to apply joint angle constraints. The joint angle limits for the two robotic arms are listed in Table 2.3.

Table 2.3. Joint angle ranges of the robotic arms.

Model	myCobot 280 Pi (degrees)	UR3 (degrees)
Joint 1	-168~+168	-360~+360
Joint 2	-135~+135	-360~+360
Joint 3	-150~+150	-360~+360
Joint 4	-145~+145	-360~+360
Joint 5	-165~+165	-360~+360
Joint 6	-180~+180	-inf~+inf

Then, the Monte Carlo method was used to create the workspace of the robotic arm. The Monte Carlo method [34] is a stochastic simulation technique based on probability and statistical theory, which utilizes random numbers to solve many computational problems. Specifically, a uniform distribution was used to randomly sample the joint angles of the robotic arm and calculate the end effector positions for each set of joint angles.

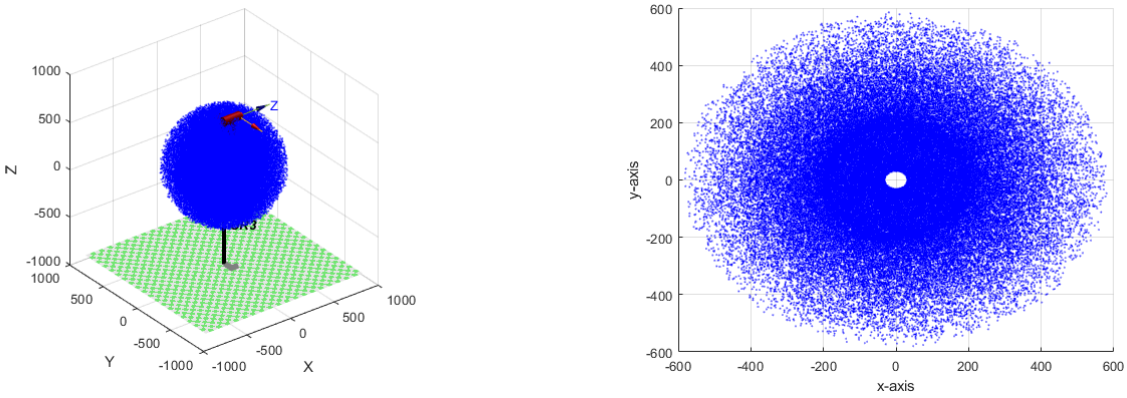
The workspaces for myCobot 280 and UR3 robotic arms are shown in Figures 2.2 and 2.3, respectively.



(a) (b)

NOTE: All units in the figure are in millimeters.

Figure 2.3. Robotic arm workspace. (a) Workspace of myCobot 280 Pi; (b) Workspace of myCobot 280 Pi on the XOY plane.



(a) (b)

NOTE: All units in the figure are in millimeters.

Figure 2.4. Robotic arm workspace. (a) Workspace of UR3; (b) Workspace of UR3 on the XOY plane.

It can be observed that the central area on the XOY plane for both collaborative robotic arms is empty, indicating positions that the robotic arms cannot reach. Therefore, when setting target points and creating trajectories, this area should be avoided.

3. MULTI-OBJECTIVE OPTIMIZATION ALGORITHMS

3.1 Introduction

In this chapter, the algorithms developed based on genetic algorithm and particle swarm optimization are introduced, and their development steps for the inverse kinematics problem are discussed. For energy-aware inverse kinematics, this thesis introduces a multi-objective optimization approach, defining multi-objective functions related to position, orientation, and total amount of joint rotations. Due to the different units and magnitudes of these objectives, a weighted sum method is used to weight the three objective functions, thus developing a weighted multi-objective genetic algorithm and a weighted multi-objective particle swarm optimization. Additionally, the application of the Pareto front in inverse kinematics problems, as well as the differences between two-objective and three-objective optimization, are discussed, providing a reference for selecting the optimal solution.

3.2 Optimization algorithms

3.2.1 Genetic algorithm

The genetic algorithm was proposed by John Holland in the 1970s. This algorithm is designed based on the evolutionary principles observed in nature. It is a computational model that simulates the process of biological evolution according to Darwin's theory of natural selection and the mechanisms of genetics. It is a method of searching for optimal solutions by mimicking the natural evolutionary process, and it belongs to the category of evolutionary algorithms [35].

This research proposed a genetic algorithm for inverse kinematics based on these five steps. Since the collaborative robotic arm used in the research has six joint angles and constraints, that is, θ_1, θ_2, θ_3, θ_4, θ_5, and θ_6 that satisfy the ranges of motion for the robotic arm joints. The established forward kinematics model is transformed into the objective function as in Equation (3.1) to obtain the homogeneous transformation matrix of the end effector. Note that the position vector is denoted as P given by Equation (3.2), and the orientation matrix is designated as O in Equation (3.3).

$$f(\theta) = FK(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6) = \begin{bmatrix} n_x & o_x & a_x & p_x \\ n_y & o_y & a_y & p_y \\ n_z & o_z & a_z & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3.1}$$

$$P = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \tag{3.2}$$

$$O = \begin{bmatrix} n_x & o_x & a_x \\ n_y & o_y & a_y \\ n_z & o_z & a_z \end{bmatrix} \tag{3.3}$$

Since there are six variables in the objective function, the chromosome consists of six genes, as shown in Figure 3.1.

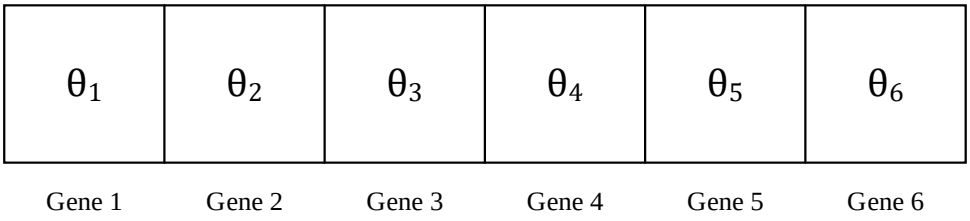


Figure 3.1. Chromosome representation in the genetic algorithm for 6-DOF robotic arms.

The genetic algorithm mathematically converts the problem-solving process into a process similar to chromosome genes in biological evolution. The genetic process includes the following three operations: crossover, mutation, and selection [36].

Initialization and definition of the fitness function are required before the genetic process begins. In initialization, the number of chromosomes in each generation needs to be determined. The chromosomes are randomly generated within joint angle constraints.

The fitness function evaluates the chromosomes in the previous initialization. By substituting θ_1, θ_2, θ_3, θ_4, θ_5, and θ_6 into the objective function, the actual position P and orientation O of the end effector are obtained. The fitness function is the weighted sum of the position error and orientation error, given in Equation (3.4), where the position error is the Euclidian distance between the actual position and the target position, defined as Equation (3.5). The orientation error

is the root of the sum of the squared differences between the elements of the actual orientation matrix and the elements of the target orientation matrix, as given by Equation (3.6).

$$fitness(c) = w_1 * positionError + w_2 * orientationError \tag{3.4}$$

$$positionError = \sqrt{(p_x - x_t)^2 + (p_y - y_t)^2 + (p_z - z_t)^2} \tag{3.5}$$

$$orientationError = \sqrt{\begin{aligned}&(n_x - n_{xt})^2 + (n_y - n_{yt})^2 + (n_z - n_{zt})^2 \\ &+(o_x - o_{xt})^2 + (o_y - o_{yt})^2 + (o_z - o_{zt})^2 \\ &+(a_x - a_{xt})^2 + (a_y - a_{yt})^2 + (a_z - a_{zt})^2\end{aligned}} \tag{3.6}$$

With initialization and determination of the fitness function, Figure 3.3 illustrates the flowchart of the genetic algorithm used in this thesis.

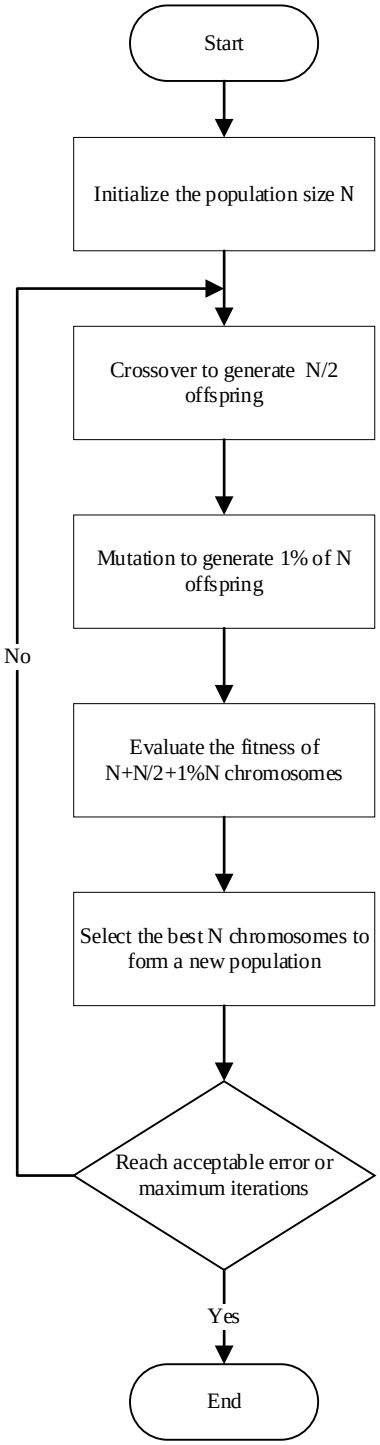


Figure 3.2. Genetic algorithm flowchart.

1. Crossover

Crossover is the main genetic operator for generating new individuals. It combines the genetic material of two parent individuals to produce offspring. The direction-based crossover operator can be used here [37], which generates a new individual $B_{offspring}$ from two parents B_1 and B_2 as shown in Figure 3.3 using Equation (3.7),

$$B_{offspring} = r \cdot (B_2 - B_1) + B_2 \tag{3.7}$$

where r is a random number between 0 and 1. Note that fitness(B_2) $\geqslant$ fitness(B_1) for maximization problems and fitness(B_2) $\leqslant$ fitness(B_1) for minimization problems. For the inverse kinematics problem, it is to minimize the position error and orientation error.

The process in Figure 3.3 demonstrates the crossover of two chromosomes to generate the offspring in general.

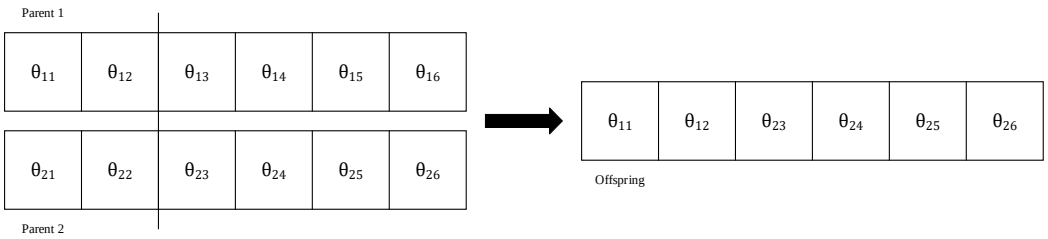


Figure 3.3. The crossover process of chromosomes.

2. Mutation

Mutation introduces random changes to the individuals to maintain genetic diversity. This can involve small adjustments to the joint angles. A certain number of mutated individuals are generated and placed into the population. These mutated individuals are generated by randomization. The mutation rate is typically set to a low percentage, in this thesis, 1%, to reflect the natural occurrence of mutations. This step helps in exploring new areas of the solution space and prevents the algorithm from getting stuck in local optima.

3. Selection

Selection is based on the fitness values of individuals, that is, sets of joint angles. The goal is to select the best-performing N individuals for the new population.

After iteration, individuals are sorted based on their fitness values, and the best individuals are retained in the population for the next iteration, repeating steps as shown in Figure 3.2. The pseudocode by using the genetic algorithm with a process of crossover and full mutation (1% of population) [36] to solve the inverse kinematics is shown as Algorithm 1.

Algorithm 1 GA for IK

Set target position P_t and target Orientation O_t
Initialize population size N, crossover rate r and mutation rate 1%
Define fitness function fitness(θ_1, …,θ_6) using forward knimatics model
Set joint angle limits
for iteration = 1 to M **do**
  for i = 1 to N/2 **do**
    $parent_1$, $parent_2$ = select from selectedParents
    $offspring_1$, $offspring_2$ = crossover($parent_1$, $parent_2$)
    add $offspring_1$ and $offspring_2$ to newPopulation
  end for
  for each i in newPopulation **do**
    mutate(i)
  end for
  for i in population do
    $P, O = FK(population(i))$
    $fitness(i) = w_1 * \sqrt{(P - P_t)^2} + w_2 * \sqrt{(O - O_t)^2}$
  end for
  Select best individuals for next generation
end for

3.2.2 Particle swarm optimization

Particle Swarm Optimization (PSO) is an evolutionary computation technique developed by James Kennedy and Russell Eberhart in 1995 [38]. The PSO algorithm was originally developed to graphically simulate the graceful and unpredictable movements of bird flocks. Through the observation of animal social behavior, it was discovered that the social sharing of information within a group provides an evolutionary advantage, and this concept served as the foundation for developing the algorithm. By incorporating velocity matching of neighbors, considering multi-dimensional search, and acceleration based on distance, the initial version of PSO was formed. Later, the introduction of inertia weight w improved the balance between exploitation and exploration, leading to a modified version of the algorithm [39]. For the inverse kinematics problem, the particle contains the joint angles of the robotic arm. Specifically, for the

six DOF collaborative robotic arm under study, there are six variables. The i[th] particle X_i can be defined in Equation (3.8). The variables in the particle x_i are also called the position of the i[th] particle.

$$X_i = \begin{bmatrix} x_{i1} & x_{i2} & x_{i3} & x_{i4} & x_{i5} & x_{i6} \end{bmatrix} \tag{3.8}$$

Each particle has a corresponding velocity, which is used to correct the particle position. The dimension of velocity is consistent with the particle, which can be written as Equation (3.9).

$$V_i = \begin{bmatrix} v_{i1} & v_{i2} & v_{i3} & v_{i4} & v_{i5} & v_{i6} \end{bmatrix} \tag{3.9}$$

The allowable upper and lower limits of each component in particle position and velocity are denoted by Equations (3.10) and (3.11), respectively.

$$x_{\min} < x_{ij} < x_{\max} \text{ for } j = 1, 2, \ldots, 6 \tag{3.10}$$

$$v_{ij} = \begin{cases} v_{\min} & if \quad v_{ij} < v_{\min} \\ v_{ij} & if \quad v_{\min} \le v_{ij} \le v_{\max} \\ v_{\max} & if \quad v_{ij} > v_{\max} \end{cases} \text{ for } j = 1, 2, \ldots, 6 \tag{3.11}$$

The velocity component can also be set according to the ratio k, as shown in Equation (3.12).

$$v_{\max} = k x_{\max} \text{ and } v_{\min} = -v_{\max} \tag{3.12}$$

There are multiple particles in a particle swarm, called the population. A population consists of H particles, that is, $[X_1 X_2 \ldots X_H]^T$.

With the above parameters, there are two iterative equations in the particle swarm optimization. First, the velocity iteration can be written as Equation (3.12), where c_1 and c_2 are the positive acceleration coefficients that drive each particle to the individual best and global best position, respectively. r_1 and r_2 are the uniformly distributed random numbers from 0 to 1. In Equation (3.12), pbest(i) is the individual best position, that is, the local optimal position; and gbest is the global best position, that is, the global optimal position. ω is the positional weight.

$$V_i = \omega V_i + c_1 r_1 (pbest_i - X_i) + c_2 r_2 (gbest - X_i) \tag{3.13}$$

The particle position iteration can be written as Equation (3.14).

$$X_i = X_i + V_i \tag{3.14}$$

Note that at each iteration, the individual best position (pbest) is obtained as follows. Each particle compares its current objective function with the best one generated by the individual best set: $pbest = [P_1 P_2 \ldots P_H]^T$. P_i of the ith particle is selected based on evaluating the fitness functions generated by the individual best P_i and the particle X_i, respectively. P_i is set to X_i when the fitness score from X_i is better than the one from P_i.

The global best (gbest) is the best one among the individual best particles: $gbest \in [P_1 P_2 \ldots P_H]^T$.

The steps of the PSO algorithm based on [40] are as follows:

1. Initialization: Initialize a swarm of particles (population size H), including random positions and velocities.

2. Fitness evaluation: Evaluate the fitness of each particle.

3. Update individual best (pbest): For each particle, compare its fitness value with the best position it has encountered (pbest). If the current value is better, update pbest to the current position.

4. Update global best (gbest): For each particle, compare its fitness value with the global best position encountered by the entire swarm (gbest). If the current value is better, update gbest to the current position.

5. Update velocity and position: Update the velocity and position of each particle according to the function.

6. Termination Check: If the termination condition is not met (usually a sufficiently good fitness value or reaching a preset maximum number of generations Gmax), return to step 2.

The pseudocode for the inverse kinematics based on PSO is shown in Algorithm 2.

Algorithm 2 PSO for IK

Set target position P_t and target Orientation O_t
Initialize particles P, velocity w, coefficients c_1 and c_2
Initialize local best positions and global best position
Initialize objective function values, and solution stores
Define fitness function (forward kinematics model) $f(\theta_1 \sim \theta_6)$
Set joint angle limits
for iteration = 1 to M, **do**
  for each particle p **do**
    $p(i) = f(\theta_i)$
    Update local best by minimizing f(local best position)
    Update global best by minimizing f(global best position)
    Update particle velocity and position
    $V_i = \omega V_i + c_1 r_1 (pbest_i - X_i) + c_2 r_2 (gbest - X_i)$
    $X_i = X_i + V_i$
  end for
end for

3.3 Multi-objective optimization

3.3.1 Objective functions and weighted sum method

The weighted multi-objective genetic and particle swarm optimization algorithms developed in this thesis are based on the genetic algorithm and particle swarm optimization [36, 40, 41], respectively.

Regarding the design of the objective function for inverse kinematics, the first objective function, f1, is set as the Euclidean distance between the actual position of the end effector and the target position, written as Equation (3.15).

$$f_1(\theta_1, ..., \theta_6) = \sqrt{(x - x_t)^2 + (y - y_t)^2 + (z - z_t)^2} \tag{3.15}$$

The second objective function, f2, is set as the orientation error of the end effector. Since the orientation representation used in this study is the orientation matrix, the error is defined as the root of the sum of the squares of the differences of the elements between the actual orientation matrix and the target orientation matrix, as in Equation (3.16).

$$f_2(\theta_1,...,\theta_6) = \sqrt{\begin{matrix}(n_x - n_{xt})^2 + (n_y - n_{yt})^2 + (n_z - n_{zt})^2 \\ +(o_x - o_{xt})^2 + (o_y - o_{yt})^2 + (o_z - o_{zt})^2 \\ +(a_x - a_{xt})^2 + (a_y - a_{yt})^2 + (a_z - a_{zt})^2\end{matrix}} \quad (3.16)$$

The third objective function, that is, the energy-aware objective function, f3, is defined as the sum of the six joint rotations as in Equation (3.17).

$$f_3(\theta_1,...,\theta_6) = \sum_{i=1}^{6} |\theta_i| \quad (3.17)$$

The weighted sum method is used in the multi-objective optimization [42]. The weighted multi-objective equations can be represented as Equation (3.18) with the condition in Equation (3.19).

$$f(\theta_1,...,\theta_6) = w_1 f_1(\theta_1,...,\theta_6) + w_2 f_2(\theta_1,...,\theta_6) + w_3 f_3(\theta_1,...,\theta_6) \quad (3.18)$$

$$w_1 + w_2 + w_3 = 1 \quad (3.19)$$

3.3.2 Weighted multi-objective genetic algorithm

With the multi-objective optimization method, the pseudocode for the weighted multi-objective genetic algorithm (WMOGA) is shown in Algorithm 3.

Algorithm 3 WMOGA for Energy-Aware IK

```
Set target position Pt and target orientation Ot
Initialize objective function values, weights and solution stores
Initialize GA parameters
Check joint range limitations
Iterate over weight combinations
for w1=initial value1
   for w3=initial value2
      w2=1-w1-w3
      if w2>0
         Execute GA to obtain θ_i (i = 1,...,6)
         Calculate position matrix P and orientation matrix O
         Calculate f1, f2, f3
         if f1 and f2<= allowable limits
            store objective function values, weights and solutions
         end if
      end if
   end for
end for
```

3.3.3 Weighted multi-objective particle swarm optimization

Similarly, the pseudocode for the weighted multi-objective particle swarm optimization (WMOPSO) is shown in Algorithm 4.

Algorithm 4 WMOPSO for Energy-Aware IK

```
Set target position Pt and target Orientation Ot
Initialize objective function values, weighs and solution stores
Initialize PSO parameters
Check joint range limitations
Iterate over weight combinations
for w1=initial value1
   for w3=initial value2
      w2=1-w1-w3
      if w2>0
         Execute PSO to obtain θ_i (i = 1,...,6)
         Calculate position matrix P and orientation matrix O
         Calculate f1, f2, f3
         if f1 and f2<= allowable limits
            store objective function values, weights and solutions
         end if
      end if
   end for
end for
```

3.4 Pareto front in energy-aware inverse kinematics

In a multi-objective optimization problem, achieving the optimal balance between conflicting objectives is important for effective decision-making. The Pareto front is a critical concept, representing the set of non-dominated solutions where no objective can be improved without degrading another [43-44].

For an inverse kinematics problem, determining the joint angles to achieve a desired end effector position and orientation, involves minimizing position and orientation errors. The two-dimensional Pareto front, as visualized in Figure 3.4(a), illustrates these balances between the first two objectives, f1 (position error) and f2 (orientation error). When energy consumption (f3) is added as the third objective function, resulting in energy-aware inverse kinematics, the Pareto front becomes three dimensions and can also be called the Pareto surface, as shown in Figure 3.4(b).

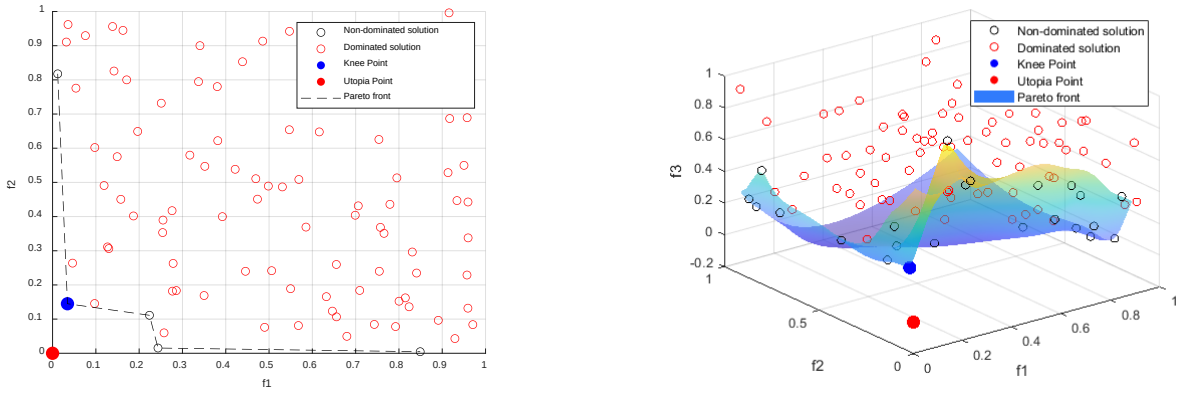


Figure 3.4. Pareto front. (a) Pareto front in two-objective optimization; (b) Pareto front in three-objective optimization (Pareto surface).

As shown in Figure 3.4, the non-dominated solutions formulate the Pareto front and reflect optimal balances among the objective function values. Dominated solutions represent inferior solutions where better alternatives exist. The knee point indicates an optimal compromise where further improvement in one objective significantly worsens the others. Utopia point represents the ideal scenario where all objectives achieve their best possible values. Therefore, the Pareto front is a method for analyzing solutions, assisting in identifying efficient balances among position errors, orientation errors, and energy consumption.

4. EXPERIMENTS AND RESULTS

4.1 Inverse kinematics solutions for single points

The experiments in this thesis are divided into two parts. The first part is that given a target point, the WMOGA and WMOPSO algorithms are applied to find the inverse kinematics solution. The second part of the experiments includes creating typical target trajectories such as circles and sine waves and then testing these trajectories on two collaborative robotic arm structures using the two proposed algorithms. Next, the trajectories are first applied in a simulation environment using MATLAB. Once the trajectories are confirmed to be accurate, the results are then validated via a real robotic arm in a real-world environment.

The experiment was first started from a given target point. For each proposed algorithm, each robotic arm was run 10 times to obtain the inverse kinematic solutions and the average errors for each objective and the sum of the joint rotations.

The spatial coordinates of the target point in the experiment are set to (150, 150, 200) in millimeters, chosen in the first quadrant that can be reached by both robotic arms, with the end effector facing downward, i.e., when the rotation matrix is $\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$. The experimental results are shown in Table 4.1. For comparison purposes, the energy-optimal joint rotations from the analytical solution for the myCobot 280 Pi and UR3 are 5.5489 rad, and 8.0103 rad, respectively. The steps for the analytical inverse kinematics solutions are provided in Appendix A.

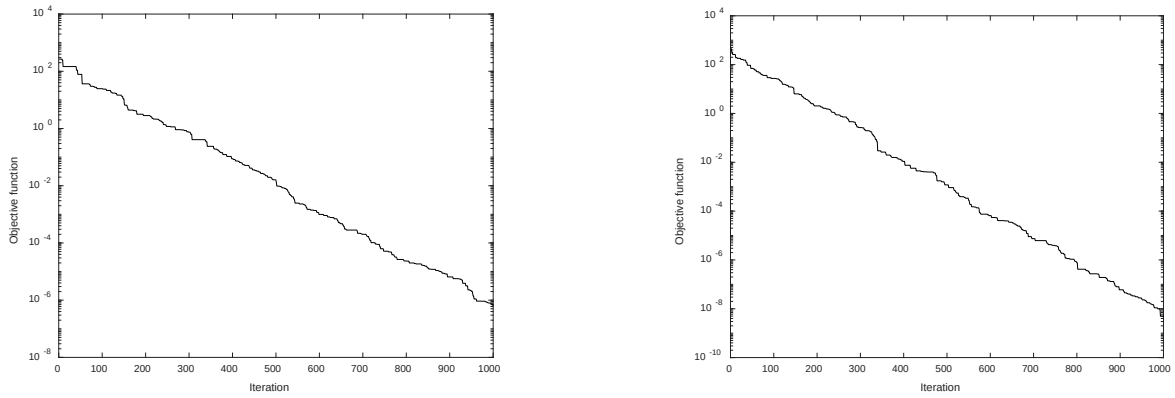
Based on the results in Table 4.1, it can be seen that the total joint rotation finally optimized by the GA and PSO based algorithms is the same for the same robotic arm at the same point. And the WMOPSO algorithm obtains smaller position and orientation errors than WMOGA. It is observed in the experiments that the position and orientation errors obtained by the WMOPSO algorithm are basically at the same order of magnitude.

Table 4.1. Performance comparison of GA, PSO, WMOGA, and WMOPSO in single point experiments (average of 10 runs).

Algorithm-model	Position error (m)	Orientation error	Joint rotations (rad)	Minimum joint rotations (rad)	Maximum joint rotations (rad)
GA-myCobot 280 Pi	2.0877×10^{-7}	7.7289×10^{-10}	4.5860	3.1416	5.5489
PSO-myCobot 280 Pi	3.5126	5.4187×10^{-7}	4.1086	3.1416	5.5489
WMOGA-myCobot 280 Pi	3.7490×10^{-12}	4.5517×10^{-11}	3.1416	3.1416	3.1416
WMOPSO-myCobot 280 Pi	0	1.5564×10^{-16}	3.1416	3.1416	3.1416
GA-UR3	3.3769×10^{-8}	1.5672×10^{-10}	8.1626	4.8687	15.8228
PSO-UR3	2.0097×10^{-14}	2.6162×10^{-16}	5.8112	4.8687	8.0103
WMOGA-UR3	1.3438×10^{-12}	2.0267×10^{-11}	4.8687	4.8687	4.8687
WMOPSO-UR3	0	3.1265×10^{-16}	4.8687	4.8687	4.8687

Most importantly, it is evidenced from Table 4.1 that WMOGA and WMOPSO algorithms produce the minimum average rotation values over the 10 runs, which are also smaller than the values of the total amount of rotations by the analytical solutions: 5.5489 rad for myCobot and 8.0103 rad for UR3 This validates the feasibility by adding the third objective function as proposed in Equation (3.17).

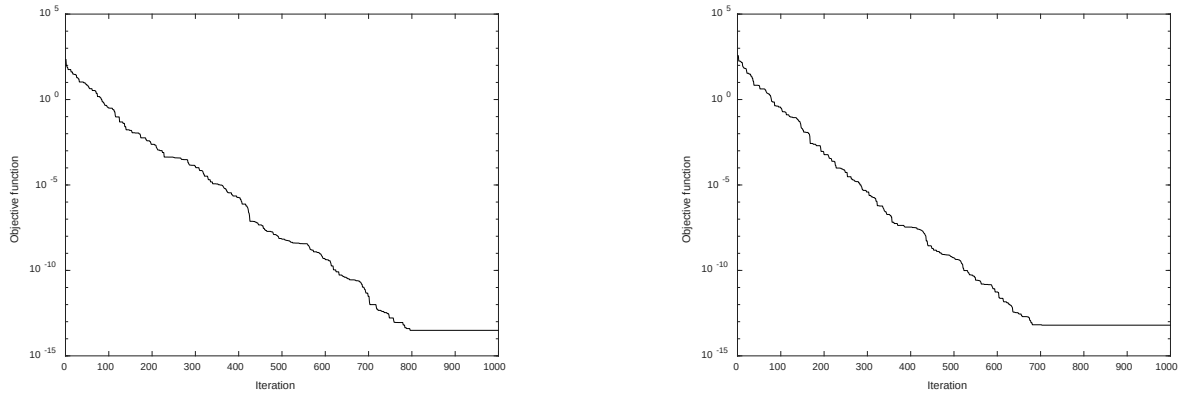
The iterations of the genetic algorithm for the two robotic arms are shown in Figure 4.1.



(a) (b)

Figure 4.1. GA iteration with position and orientation objective functions. (a) myCobot 280 Pi; (b) UR3.

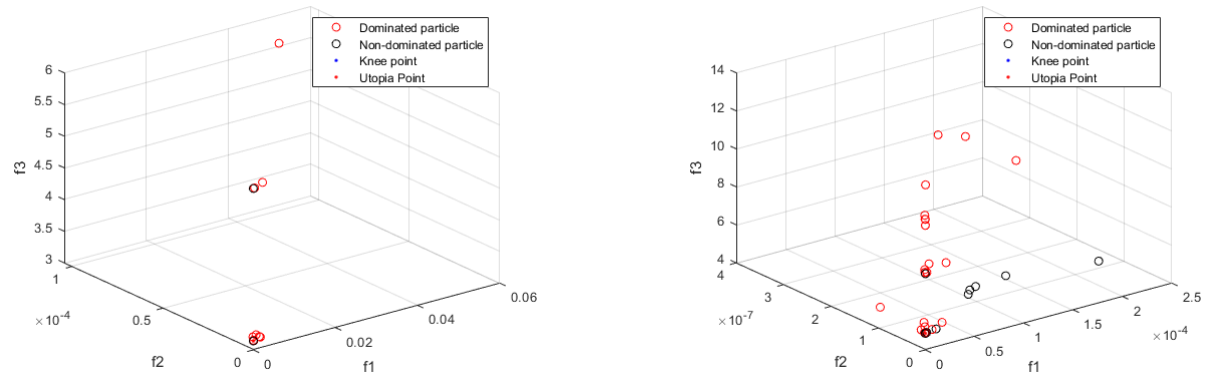
The iterations of the PSO algorithm for the two robotic arms are shown in Figure 4.2.



(a) (b)

Figure 4.2. PSO iteration with position and orientation objective functions. (a) myCobot 280 Pi; (b) UR3.

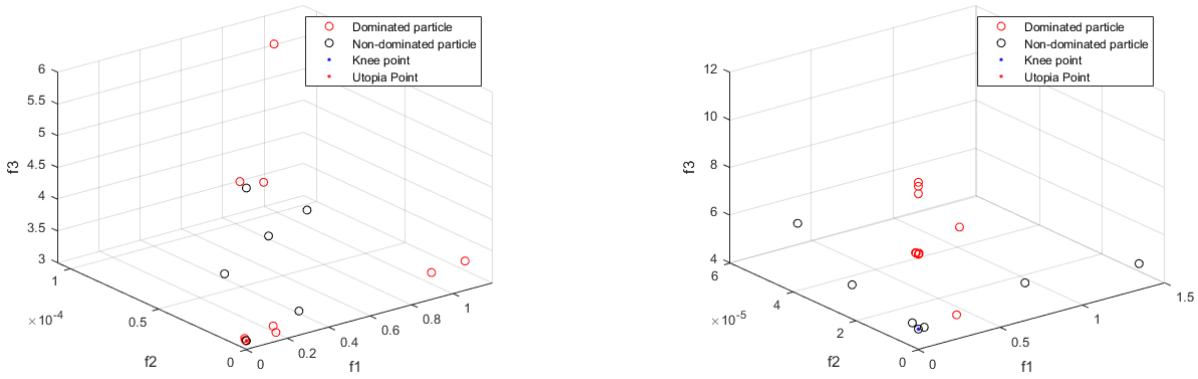
The Pareto fronts of WMOGA for myCobot 280 Pi and UR3robotic arms are shown in Figure 4.3 (a) and (b), respectively.



(a) (b)

Figure 4.3. WMOGA Pareto fronts. (a) myCobot 280 Pi; (b) UR3.

The Pareto front from the WMOPSO algorithm for the two robotic arms are shown in Figure 4.4 (a) and (b), respectively.



(a) (b)

Figure 4.4. WMOPSO Pareto fronts. (a) myCobot 280 Pi; (b) UR3.

4.2 Inverse kinematics solutions for trajectories

In this group of experiments, three trajectories were adopted for simulation and validation. The first one is a circular trajectory with the end effector of the robotic arm vertically pointing downward. As in most application scenarios, the gripper on the robotic arm is orientated downward to complete the pick and place.

In order to test the performance of the algorithm at a different gripper orientation, the second trajectory is selected as the same circle with the end effector 45 degrees diagonally pointing downward.

The third trajectory is a sine wave varying in x, y, and z coordinates, representing a spatial trajectory.

The positions, sizes, and other information for these three trajectories are listed in Table 4.2.

Table 4.2. Trajectory information.

Trajectory	Center/Start point (x, y, z) for myCobot 280 Pi (mm)	Center/Start point for UR3 (mm)	Size (mm)	Number of Points	Fixed Orientation
Circle	(100, 100, 200)	(200, 200, 200)	r=50	50	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$
Circle/	(150, 150, 200)	(200, 200, 200)	r=50	50	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}$
Sine	(100, 100, 200)	(200, 200, 200)	range=100, A=100, f=1/100, φ=2	50	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$

NOTE: The "/" indicates the end effector with 45 degrees downward.

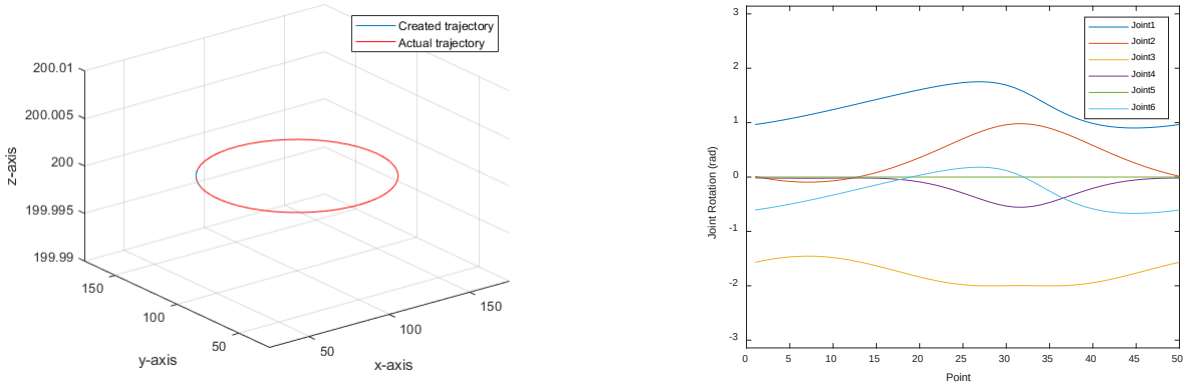
The results of the trajectory experiments are shown in Table 4.3. The trajectory experiments apply the two algorithms that performed better in the single point experiments to both robotic arms, obtaining the root mean square error of position and orientation as well as the total joint rotations. The goal of the performance metrics is to minimization.

Table 4.3. Performance comparison of WMOGA and WMOPSO in trajectory experiments.

Algorithm		WMOGA			WMOPSO		
Model	Trajectory	RMSE of position (m)	RMSE of orientation	Joint rotations (rad)	RMSE of position (m)	RMSE of orientation	Joint rotations (rad)
myCobot 280 Pi	Circle	4.0583×10^{-11}	8.4286×10^{-10}	3.9725	1.9221×10^{-8}	1.3275×10^{-8}	3.9725
	Circle/	1.7253×10^{-10}	6.1786×10^{-10}	5.0374	1.7154×10^{-6}	3.7737×10^{-8}	5.0374
	Sine	1.5665×10^{-11}	2.7856×10^{-10}	4.9485	5.4239×10^{-17}	2.4573×10^{-16}	4.9485
UR3	Circle	2.7233×10^{-11}	6.0936×10^{-11}	4.3980	5.1787×10^{-18}	2.0718×10^{-16}	4.3980
	Circle/	8.2873×10^{-12}	1.3754×10^{-10}	6.1968	4.7218×10^{-18}	2.3859×10^{-16}	6.1968
	Sine	2.7190×10^{-12}	3.4270×10^{-11}	5.1709	4.7388×10^{-18}	2.0603×10^{-16}	5.1709

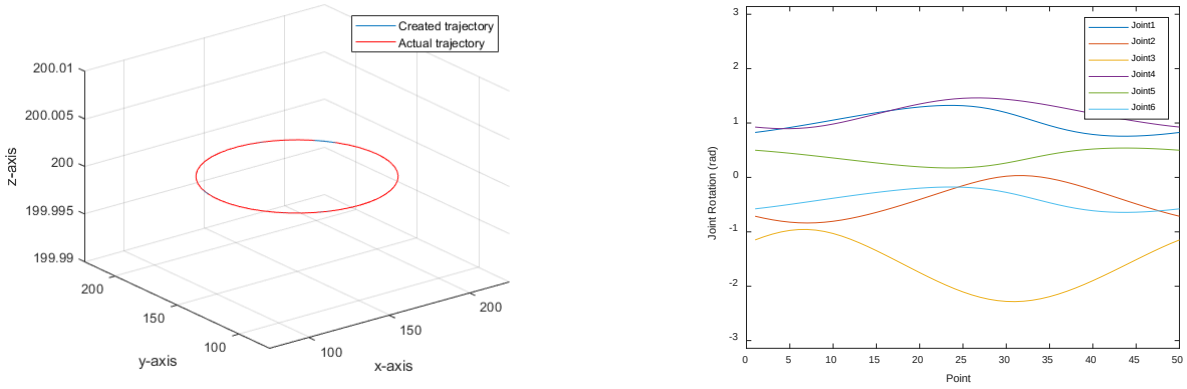
The performances of WMOGA for the myCobot 280 Pi on three trajectories are shown in Figures 4.5 to 4.7, respectively while the performances of WMOPSO for the myCobot 280 Pi on three trajectories are shown in Figures 4.8 to 4.10. Again, the performances of WMOGA for the

UR3 on three trajectories are displayed in Figures 4.11 to 4.13, respectively, and the performances of WMOPSO for the UR3 on three trajectories are depicted in Figures 4.14 to 4.16.



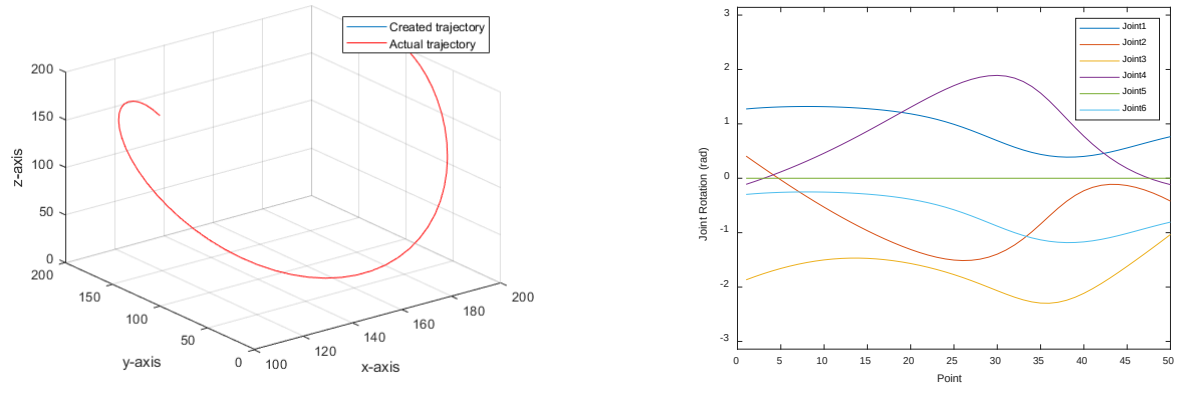
(a) (b)

Figure 4.5. The performances of the WMOGA for the myCobot 280 Pi on a circular trajectory. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.



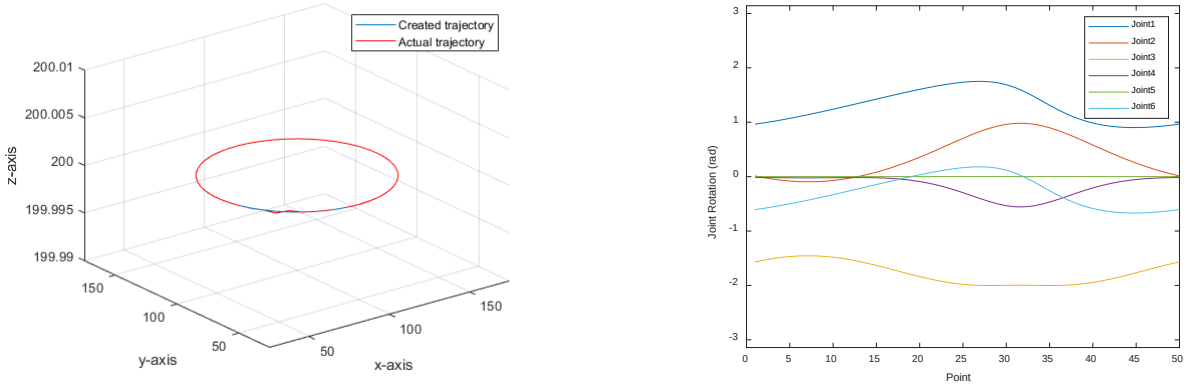
(a) (b)

Figure 4.6. The performance of the WMOGA for the myCobot 280 Pi on a circular trajectory with angles. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.



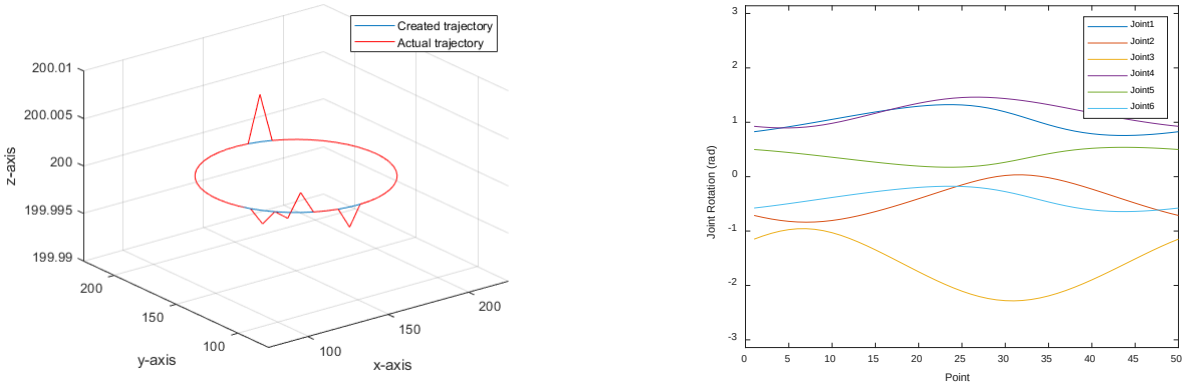
(a) (b)

Figure 4.7. The performance of the WMOGA for the myCobot 280 Pi on a sine trajectory. (a) Comparison between the target trajectory and the actual trajectory. (b) Joint angles versus trajectory points.



(a) (b)

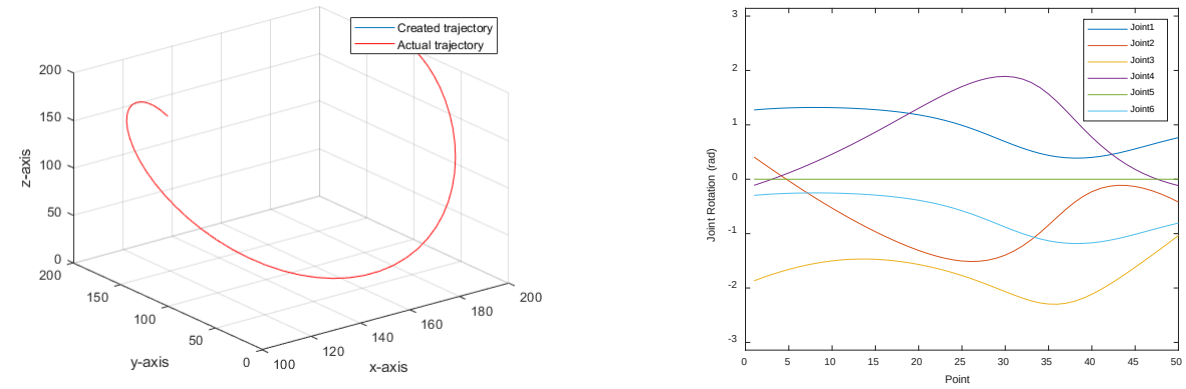
Figure 4.8. The performance of the WMOPSO for the myCobot 280 Pi on a circular trajectory. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.



(a) (b)

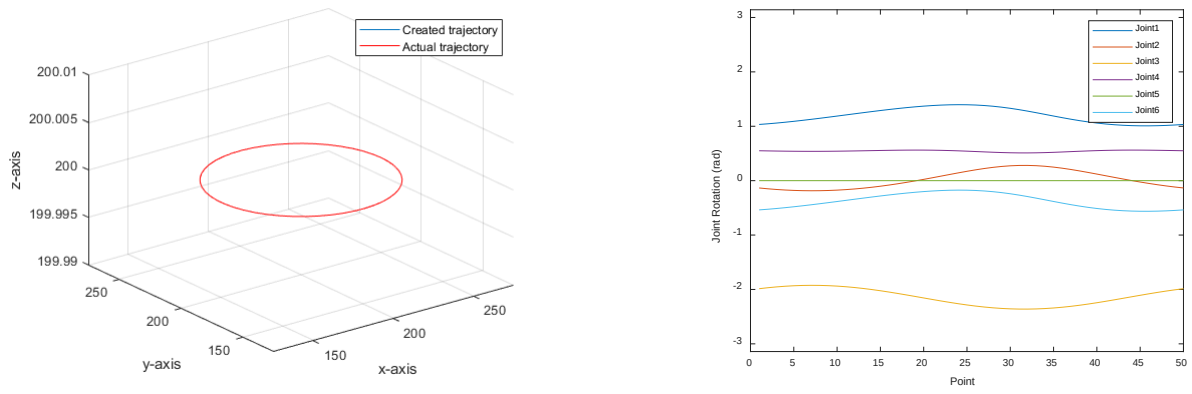
Figure 4.9. The performance of the WMOPSO for the myCobot 280 Pi on a circular trajectory with angles. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.

Note that practically, the position errors in Figure 4.9 (a) are still acceptable.



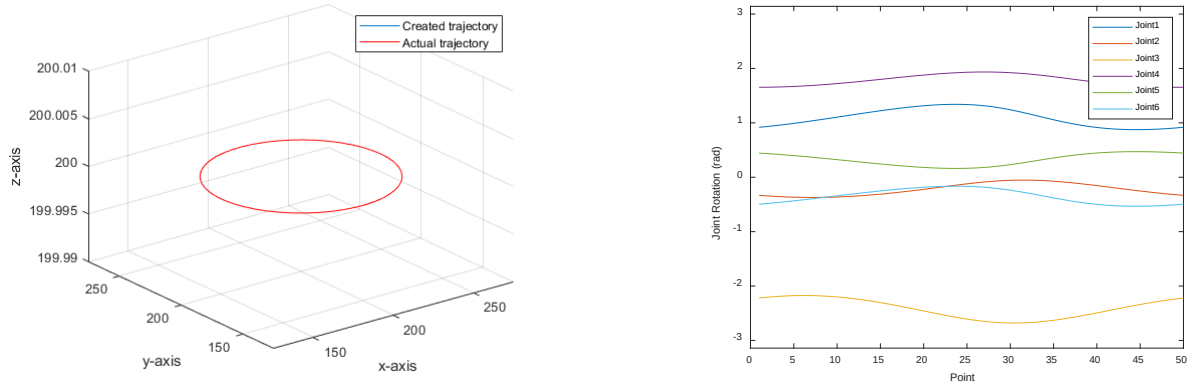
(a) (b)

Figure 4.10. The performance of the WMOPSO for the myCobot 280 Pi on a sine trajectory. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus tractor points.



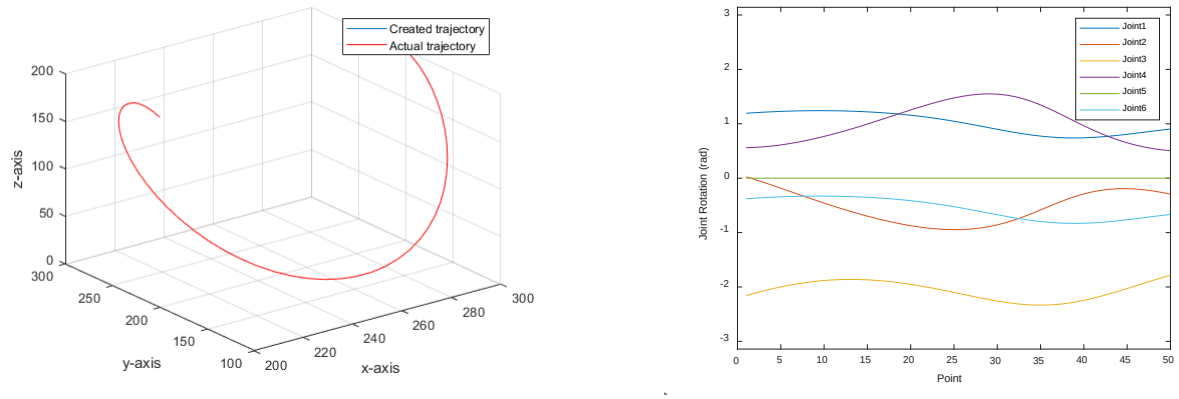
(a) (b)

Figure 4.11. The performance of the WMOGA for the UR3 on a circular trajectory. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.



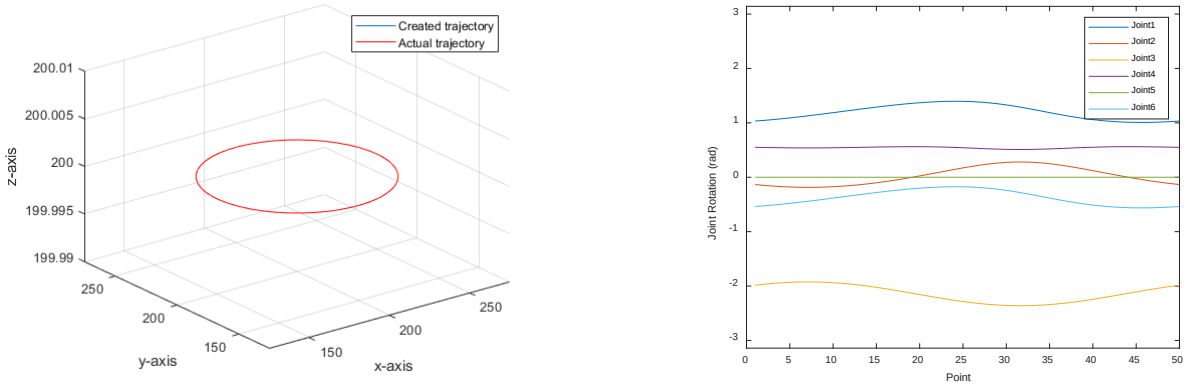
(a) (b)

Figure 4.12. The performance of the WMOGA for the UR3 on a circular trajectory with angles. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.



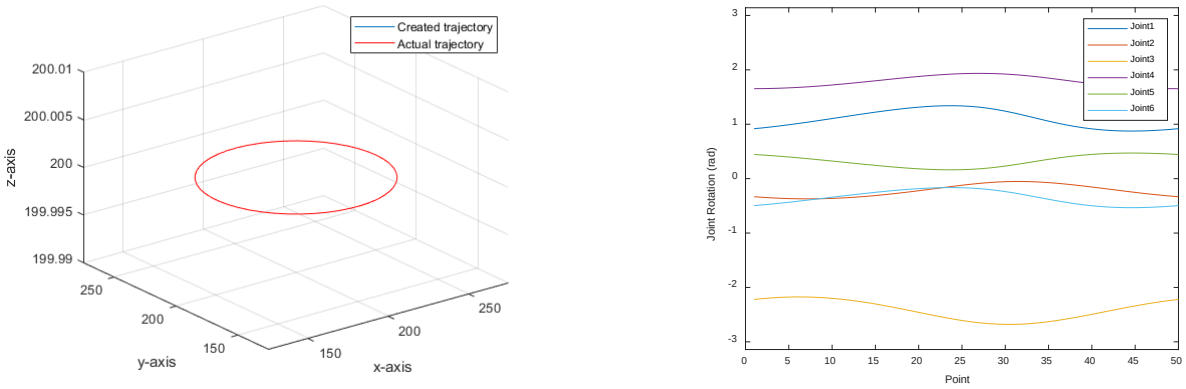
(a) (b)

Figure 4.13. The performance of the WMOGA for the UR3 on a circular trajectory. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.



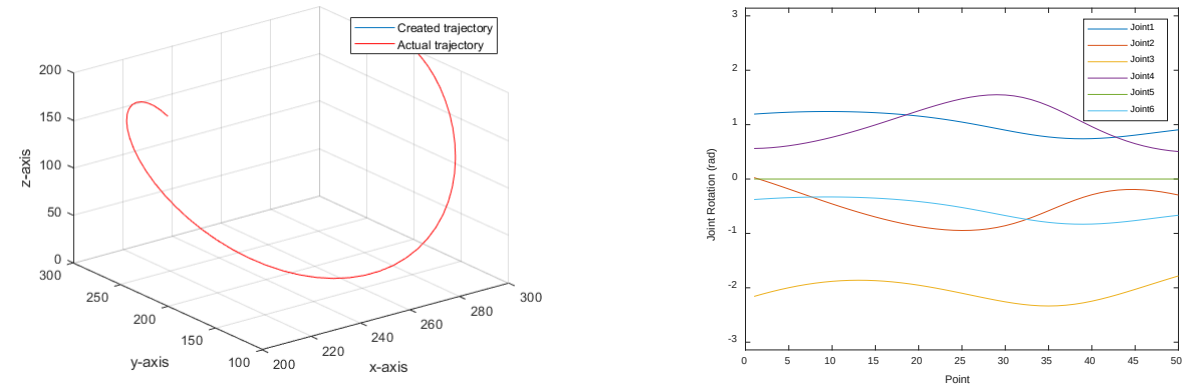
(a) (b)

Figure 4.14. The performance of the WMOPSO for the UR3 on a circular trajectory. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.



(a) (b)

Figure 4.15. The performance of the WMOPSO for the UR3 on a circular trajectory with angles. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.



(a) (b)

Figure 4.16. The performance of the WMOPSO for the UR3 on a sine trajectory. (a) Comparison between the target trajectory and the actual trajectory; (b) Joint angles versus trajectory points.

The evaluation of WMOGA and WMOPSO algorithms on the myCobot 280 Pi and UR3 robotic arms showed that WMOPSO generally achieved lower RMSE in both position and orientation across all tested trajectories. Additionally, the WMOPSO algorithm demonstrated better energy-aware performance, indicated by lower total joint rotations, making it a superior choice for applications requiring high precision and energy awareness in robotic systems.

Finally, a comparison of the developed WMOGA and WMOPSO algorithms on the circular trajectory with a multi-objective modified PSO (MO-M-PSO) algorithm in [12] is listed in Table 4.4. The developed multi-objective based algorithms outperform the one in the reference [12].

Table 4.4. Performance comparison between the developed algorithms and existing algorithm in the reference.

Algorithm	Position error (m)	Orientation error
MO-M-PSO (Best) [12]	2.31×10^{-8}	1.85×10^{-8}
WMOGA (Mean)	3.3908×10^{-11}	6.0936×10^{-11}
WMOPSO (Mean)	9.6105×10^{-9}	6.6375×10^{-9}

4.3 Simulation verification

After obtaining the inverse kinematics solutions for the different trajectories, the inverse solutions were first simulated using the robotics toolbox. And the trails of the robotic arm operation were visualized and the experimental results for each trajectory group are shown in Figures 4.16 to 4.20, respectively.

(a) (b)

Figure 4.17. Simulation of circular trajectories on myCobot 280 Pi. (a) WMOGA; (b)WMOPSO.

(a) (b)

Figure 4.18. Simulation of circular trajectories each with an orientation angle of 45 degrees downward on myCobot 280 Pi. (a) WMOGA; (b) WMOPSO.

(a) (b)

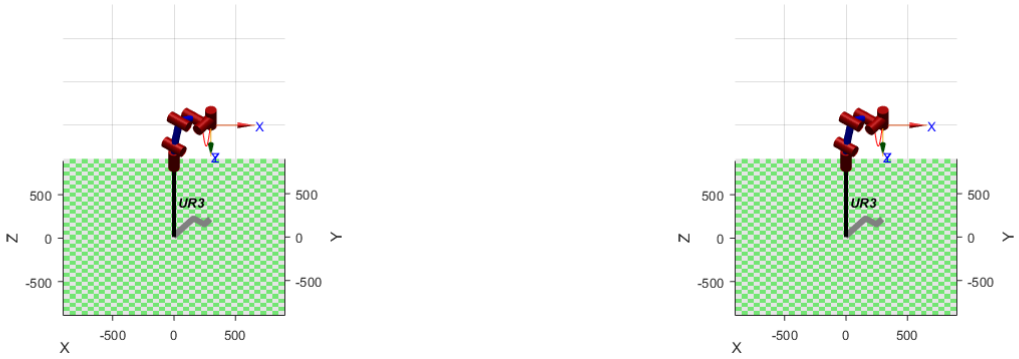
Figure 4.19. Simulation of sine trajectories on myCobot 280 Pi. (a) WMOGA; (b) WMOPSO.

(a) (b)

Figure 4.20. Simulation of circular trajectories on UR3. (a) WMOGA; (b) WMOPSO.

(a) (b)

Figure 4.21. Simulation of circular trajectories each with an orientation angle of 45 degrees downward on UR3. (a) WMOGA; (b) WMOPSO.



(a) (b)

Figure 4.22. Simulation of sine trajectories on UR3. (a) WMOGA; (b) WMOPSO.

4.4 Real robotic arm implementation

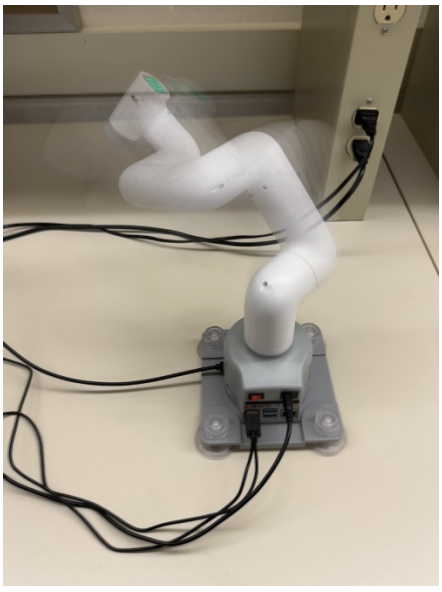
Finally, the inverse kinematics solutions for the three sets of trajectories were validated on the myCobot 280 Pi cobot in the laboratory.

Figures 4.23 to 4.25 show the real robotic arm validation of different trajectories obtained using the proposed WMOGA and WMOPSO algorithms.

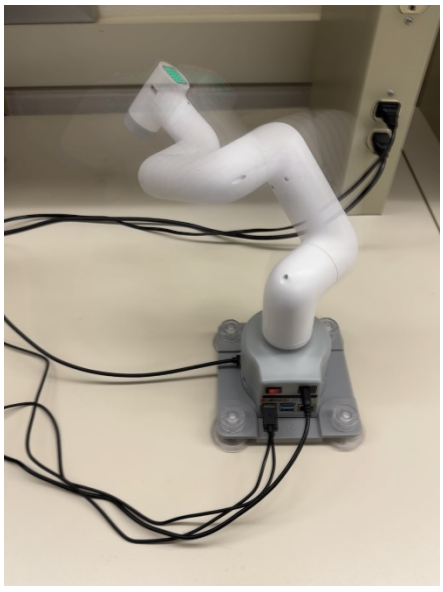
(a)

(b)

Figure 4.23. Real-world robotic arm validation or circular trajectories. (a) WMOGA; (b) WMOPSO.

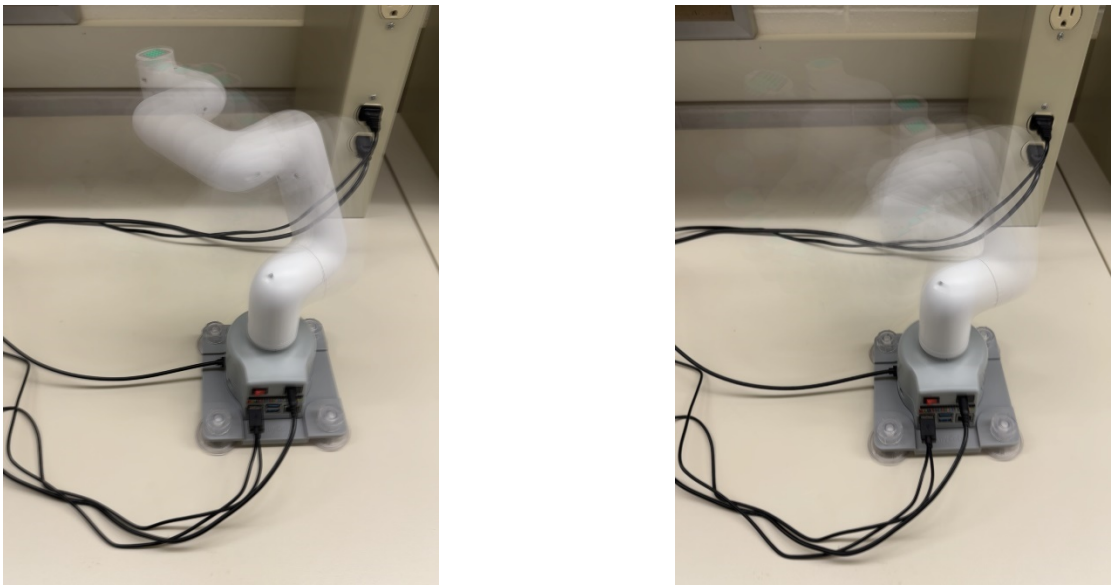


(a)



(b)

Figure 4.24. Real-world robotic arm validation of circular trajectories with angled end effector. (a) WMOGA; (b) WMOPSO.



(a) (b)

Figure 4.25. Real-world robotic arm validation of sine trajectories. (a) WMOGA; (b) WMOPSO.

Our experiments demonstrated that the robotic arm follows the desired trajectories.

5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

This thesis addressed the inverse kinematics problem of robotic arms using an energy-aware approach through multi-objective optimization by considering that the energy consumption of the robot is proportional to the total amount of joint rotations to the target position. Specifically, during the inverse kinematics optimization process, the total amount of joint rotations, position error, and orientation error are considered as objective functions. The algorithms developed use a weighted sum of the multiple objective functions to iterate over a certain range of weights to obtain the inverse kinematics solution within the required error range. The experimental results show that the obtained inverse kinematics solutions have a minimal error with the target points and orientations, and the joints maintain small rotations, resulting in very smooth motion curves for the robotic arm.

The experiments also demonstrate that as long as the trajectories are within the workspace of the robotic arm, the algorithm can quickly converge while meeting acceptable error conditions. This adaptability is beneficial for manufacturing industries with varying requirements for precision and speed in different tasks.

Furthermore, the proposed algorithm can be applied to a robotic model with any number of degrees of freedom. The trajectories solved by the proposed WMOGA and WMOPSO algorithms are verified by simulations and validated by the physical robotic arm, The achieved inverse kinematic solutions demonstrate a promising performance and have potential for applications of the inverse kinematics of robotics.

5.2 Future work

There are several directions for future research on this topic. Currently, the energy awareness function is simply the sum of rotations of all joints of the robotic arm. The dynamics in robotics can be further investigated. Secondly, only two 6-DOF collaborative robotic arms were used in the study. More robotic arm structures can also be modeled and solved, such as centrally symmetric centrosymmetric industrial robotic arms, low DOF robotic arms, higher DOF redundant robotic arms, and heterogeneous robots. Moreover, during the implementation process, it was

found that the time interval between two consecutive trajectory points from the inverse kinematics solutions affects the smoothness of the robotic arm operation. This may require using motor control strategies for the robotic arm, such as proportional-integration-differential (PID) control, or our published method, the fractional-order PID (FOPID) control using reinforcement learning [45] to assist optimization and achieve smoother robot control.

APPENDIX A. ANALYTICAL SOLUTIONS FOR 6-DOF INVERSE KINEMATICS

$$n = p_x - d_6 a_x$$

$$m = p_y - d_6 a_y$$

$$u = \pm\sqrt{m^2 + n^2 - d_4^2}$$

$$\theta_1 = \tan^{-1}\left(\frac{(m/n) - (d_4/u)}{1 + (m/n)(d_4/u)}\right), \text{ two possible solutions}$$

$$\theta_6 = \tan^{-1}\left(\frac{C_1 o_y - S_1 o_x}{S_1 n_x - C_1 n_y}\right)$$

$$\theta_5 = \sin^{-1}(C_1 a_y - S_1 a_x)$$

$$\theta_2 + \theta_3 + \theta_4 = \tan^{-1}\left(\frac{C_1 S_6 n_x + S_1 S_6 n_y + C_1 C_6 o_x + S_1 C_6 o_y}{-(S_6 n_z + C_6 o_z)}\right)$$

$$r_{14} = nC_1 + mS_1 + d_5 S_{234}$$

$$r_{34} = p_z - d_6 a_z - d_1 - d_5 C_{234}$$

$$\theta_3 = \pm\cos^{-1}[(r_{14}^2 + r_{34}^2 - a_3^2 - a_2^2)/(2a_3 a_2)], \text{ two possible solutions}$$

$$\theta_2 = \tan^{-1}\left(\frac{-(r_{14}(a_2 + a_3 C_3) + r_{34} a_3 S_3)}{r_{34}(a_2 + a_3 C_3) - r_{14}(a_3 S_3)}\right)$$

$$\theta_4 = (\theta_2 + \theta_3 + \theta_4)^* - \theta_2 - \theta_3$$

REFERENCES

1. Tesla, Tesla AI Day 2022, https://www.youtube.com/watch?v=ODSJsviD_SU.

2. G. Agrim, S. Silvio, G. Surya, and F.-F. Li, "Embodied Intelligence via learning and evolution," *Nature Communications*, vol. 12, no. 1, Oct. 2021. doi:10.1038/s41467-021-25874-z.

3. "NIHF inductee George Devol invented the Industrial Robot," NIHF Inductee George Devol Invented the Industrial Robot, https://www.invent.org/inductees/george-devol.

4. "Mechanical Advantage Two Northwestern University engineers are developing cobots -- machines that, unlike robots, cooperate with workers without displacing them," Chicago Tribune, https://peshkin.mech.northwestern.edu/cobot/chitrib/jonvan.html.

5. "US5952796A - Cobots," Google Patents, https://patents.google.com/patent/US5952796A/en.

6. S. Hand, "A brief history of collaborative robots," Material Handling and Logistics, https://www.mhlnews.com/technology-automation/article/21124077/a-brief-history-of-collaborative-robots.

7. K. P. Hawkins, "Analytic inverse kinematics for the universal robots UR-5/UR-10 arms," GT Digital Repository, https://repository.gatech.edu/entities/publication/5fe1645c-bf7d-49ac-8c0e-6516603091eb.

8. "Inverse Kinematics Algorithms," MathWorks, https://www.mathworks.com/help/robotics/ug/inverse-kinematics-algorithms.html.

9. H. Badreddine, S. Vandewalle, and J. Meyers, "Sequential quadratic programming (SQP) for optimal control in direct numerical simulation of turbulent flow," *Journal of Computational Physics*, vol. 256, pp. 1–16, Jan. 2014. doi:10.1016/j.jcp.2013.08.044.

10. T. Sugihara, "Solvability-unconcerned inverse kinematics by the levenberg–Marquardt method," *IEEE Transactions on Robotics*, vol. 27, no. 5, pp. 984–991, Oct. 2011. doi:10.1109/tro.2011.2148230.

11. F. Maric et al., "Riemannian optimization for distance-geometric inverse kinematics," *IEEE Transactions on Robotics*, vol. 38, no. 3, pp. 1703–1722, Jun. 2022. doi:10.1109/tro.2021.3123841.

12. N. Rokbani, B. Neji, M. Slim, S. Mirjalili, and R. Ghandour, "A multi-objective modified PSO for inverse kinematics of a 5-DOF robotic arm," *Applied Sciences*, vol. 12, no. 14, p. 7091, Jul. 2022. doi:10.3390/app12147091.

13. H. Z. Khaleel and A. J. Humaidi, “Towards accuracy improvement in solution of inverse kinematic problem in redundant robot: A comparative analysis,” *International Review of Applied Sciences and Engineering*, vol. 15, no. 2, pp. 242–251, Jun. 2024. doi:10.1556/1848.2023.00722.

14. S. Luo, D. Chu, Q. Li, and Y. He, “Inverse kinematics solution of 6-DOF manipulator based on multi-objective full-parameter optimization PSO algorithm,” *Frontiers in Neurorobotics*, vol. 16, Mar. 2022. doi:10.3389/fnbot.2022.791796.

15. A. Malik, Y. Lischuk, T. Henderson, and R. Prazenica, “A deep reinforcement-learning approach for inverse kinematics solution of a high degree of Freedom Robotic manipulator,” *Robotics*, vol. 11, no. 2, p. 44, Apr. 2022. doi:10.3390/robotics11020044.

16. S. Zhang, Q. Xia, M. Chen, and S. Cheng, “Multi-objective optimal trajectory planning for robotic arms using Deep Reinforcement Learning,” *Sensors*, vol. 23, no. 13, p. 5974, Jun. 2023. doi:10.3390/s23135974.

17. S. Wells, “Generative AI’s Energy Problem Today is foundational,” IEEE Spectrum, https://spectrum.ieee.org/ai-energy-consumption.

18. K. Lottick, S. Susai, S. A. Friedler, and J. P. Wilson, “Energy usage reports: environmental awareness as part of algorithmic accountability”, 2019, *arXiv:1911.08354v2*.

19. V. Bolón-Canedo, L. Morán-Fernández, B. Cancela, and A. Alonso-Betanzos, “A review of Green Artificial Intelligence: Towards a more sustainable future,” *Neurocomputing*, vol. 599, p. 128096, Sep. 2024. doi:10.1016/j.neucom.2024.128096.

20. Energy-aware process scheduling in linux, https://hotcarbon.org/assets/2024/pdf/hotcarbon24-final29.pdf.

21. J. Lansky et al., “An energy-aware routing method using Firefly algorithm for flying ad hoc networks,” *Scientific Reports*, vol. 13, no. 1, Jan. 2023. doi:10.1038/s41598-023-27567-7.

22. Y. Liang and Y. Feng, “An energy-aware routing algorithm for heterogeneous wireless sensor networks,” *2009 Ninth International Conference on Hybrid Intelligent Systems*, pp. 275–278, 2009. doi:10.1109/his.2009.167.

23. J. Wittmann, D. Hornung, K. Griesbauer, and D. Rixen, “Energy-aware hierarchical control of joint velocities,” *Journal of Intelligent & Robotic Systems*, vol. 110, no. 4, Oct. 2024. doi:10.1007/s10846-024-02182-4.

24. A. Munir, A. Dutta, and R. Parasuraman, “Energy-aware coverage planning for heterogeneous multi-robot system”,2024, *arXiv:2411.02230*.

25. S. B. Niku, *Introduction to Robotics: Analysis, Control, Applications*. Hoboken, NJ: Wiley, 2020.

26. J. J. Craig, *Introduction to Robotics: Mechanics and Control*. Harlow, United Kingdom: Pearson, 2017.

27. “myCobot 280 pi (2023),” Introduction · GitBook, https://docs.elephantrobotics.com/docs/mycobot_280_pi_en/.

28. “Universal robots - DH parameters for calculations of kinematics and dynamics,” Collaborative robotic automation, https://www.universal-robots.com/articles/ur/application-installation/dh-parameters-for-calculations-of-kinematics-and-dynamics/.

29. E. Guizzo, “Universal Robots UR3 arm is small and nimble, helps to build copies of itself,” IEEE Spectrum, https://spectrum.ieee.org/universal-robots-ur3-robotic-arm.

30. E. Trucco and A. Verri, *Introductory Techniques for 3-D Computer Vision*. Upper Saddle River, NJ: Prentice Hall, 1998.

31. Y. Tang et al., “ETA-IK: Execution-Time-Aware Inverse Kinematics for Dual-Arm Systems,” 2024, arXiv:2411.14381.

32. “Robotics toolbox,” Peter Corke, https://petercorke.com/toolboxes/robotics-toolbox/.

33. P. I. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB®*. Cham, Switzerland: Springer, 2017.

34. M. H. Kalos and P. A. Whitlock, *Monte Carlo Methods*. Weinheim: Wiley-VCH, 2008.

35. A. Petrowski and S. Ben-Hamida, *Evolutionary Algorithms*. Somerset: Wiley-ISTE, 2017.

36. S.-T. Tzeng and H.-C. Lu, “Genetic algorithm approach for designing higher-order digital differentiators,” *Signal Processing*, vol. 79, no. 2, pp. 175–186, Dec. 1999. doi:10.1016/s0165-1684(99)00091-2.

37. Z. Michalewicz, L. Thomas, and S. Swarnalatha, “Evolutionary operators for continuous convex parameter spaces,” *Proceedings of the 3rd Annual conference on Evolutionary Programming*, 1994, pp. 84–97.

38. J. Kennedy and R. Eberhart, “Particle swarm optimization,” *Proceedings of ICNN’95 - International Conference on Neural Networks*, vol. 4, pp. 1942–1948. doi:10.1109/icnn.1995.488968.

39. Y. Shi and R. Eberhart, “A modified particle swarm optimizer,” *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98TH8360)*. doi:10.1109/icec.1998.699146.

40. W.-D. Chang and D.-M. Chang, “Design of a higher-order digital differentiator using a particle swarm optimization approach,” *Mechanical Systems and Signal Processing*, vol. 22, no. 1, pp. 233–247, Jan. 2008. doi:10.1016/j.ymssp.2007.08.010.

41. C. A. C. Coello, G. T. Pulido, and M. S. Lechuga, “Handling multiple objectives with particle swarm optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 8, no. 3, pp. 256–279, Jun. 2004. doi:10.1109/tevc.2004.826067.

42. Lecture 9: Multi-objective optimization, https://engineering.purdue.edu/~sudhoff/ee630/Lecture09.pdf.

43. K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester: Wiley, 2009.

44. I. Das and J. E. Dennis, “Normal-boundary intersection: A new method for generating the pareto surface in nonlinear multicriteria optimization problems,” *SIAM Journal on Optimization*, vol. 8, no. 3, pp. 631–657, Aug. 1998. doi:10.1137/s1052623496307510.

45. X. Zhou, G. Zhu, and L. Tan, “Reinforcement learning assisted design for FOPID control,” *2024 IEEE International Conference on Electro Information Technology (eIT)*, pp. 663–667, May 2024. doi:10.1109/eit60633.2024.10609906.

PUBLICATIONS

1. X. Zhou, G. Zhu, and L. Tan, “Reinforcement learning assisted design for FOPID control,” *2024 IEEE International Conference on Electro Information Technology (eIT)*, pp. 663–667, May 2024. doi:10.1109/eit60633.2024.10609906.

2. S. Mallineni, J. Moreland, G. Page, M. Schwentor, K. Toth, C. Zhou, and X. Zhou, “Hazard recognition scenario builder for on-site customizable virtual training,” *AISTech 2024 Proceedings*, pp. 30–38, 2024. doi:10.33313/388/003.

3. N. Gregurich, K. Toth, L. Yakovleva, A. Zafar, R. Zaman, C. Zhou, and X. Zhou, “Digital twin software for a continuous caster,” *AISTech 2023 Proceedings*, 2023. doi:10.33313/387/246.